UNIVERSITY OF SALERNO

Computer Science Department

Master of Science in Computer Science

MASTER THESIS

Serverless CRATOR: A serverless crawler for dark-web

SUPERVISOR

Prof. Fabio Palomba

University of Salerno

CO-SUPERVISOR

Doc. Jessica de Pascale

Tilburg University

CANDIDATE

Adriano Emanuele Califano

Enrollment Number: 0522501790

Academic Year 2024-2025

This thesis has been realized in

"I wave goodbye to the end of beginning" - DJO

Abstract

The Dark Web, a section of the World Wide Web that can only be accessed through specific software, is a critical environment for cybersecurity due to the anonymity it provides, often facilitating illegal activities and the proliferation of illegal marketplaces. Monitoring and exploring these networks is essential for intelligence and crime prevention, but requires crawling tools capable of handling the dynamic and often unstable nature of such sites. This thesis presents the design and development of a distributed serverless architecture for crawling the dark web through the Tor network. The main objective is to overcome the limitations of on-premise crawling solutions by leveraging the advantages of cloud computing, such as elastic scalability, auto-scaling, and the resilience of serverless functions. The proposed architecture, implemented on the Microsoft Azure platform, uses components such as Azure Functions for crawling logic, a durable orchestrator for long-term workflow management and a containerized Tor proxy to ensure anonymity. The system was tested on the "Cocoriko-Market" marketplace, evaluating its performance through metrics of coverage, time efficiency, and robustness. The experimental results show that, although on-premise solutions maintain an advantage in pure speed, the serverless approach offers excellent horizontal scalability and significant cost savings for intermittent or research workloads.

Contents

List of Figures

List of Tables

List of Listings

CHAPTER 1

Introduction

1.1 Application Background

In recent decades, the Web has established itself as a major global source of information, communication, and data exchange. Originally created to facilitate document sharing via hyperlinks [1], it has evolved into a complex and layered ecosystem, growing rapidly in terms of content and functionality. The visible part commonly accessible by users, known as the **surface-web**, is actually only a small fraction of the entire available information space [2]. Underneath it extends the **deep-web**, made up of content not indexed by traditional search engines, such as databases, authentication-protected pages, or dynamic content. Even more obscure is the portion known as the **dark-web**, a sub-section intentionally not indexed and accessible only through specialised technologies, such as the Tor network, which ensure anonymity and confidentiality.

Web crawling plays a central role in this context. **Crawling** [3] is the automated process of exploring online resources, aimed at collecting and organizing data from the web. This practice is the basis of many fundamental services, including content indexing by search engines, academic research, and market analysis. In other words, crawling enables the **extraction of knowledge** from large volumes of distributed

data, facilitating in-depth and often real-time analysis.

The exploration of web pages is often combined with **web scraping** [3], i.e., the automated extraction of information content present within the HTML pages visited. Web scrapers, therefore, **identify and isolate relevant data** such as texts, images, tables, or links, structuring them in formats useful for processing and analysis, such as JSON or CSV. While crawling allows you to discover new resources by navigating through hyperlinks, scraping allows you to systematically acquire the information contained in them. The integration between crawling and scraping, therefore, constitutes a powerful strategy for collecting, processing, and analyzing large amounts of data present on the Web.

However, applying web crawling techniques to the dark web entails a number of significant challenges. Resources accessible through the Tor network are often **unstable**, characterized by **high response times** and **inconsistent structures**. In addition to these difficulties, there are concerns related to the respect of anonymity, poor visibility of relationships between sites, and the sensitive nature of the content handled, which require a careful approach from both a technical and ethical point of view.

1.2 Motivations and Goals

In this context, this work aims to explore the possibility of developing a serverless crawling system for the Tor environment, starting from architectures that already exist in the domain of serverless crawling and adapting them to the specific features and challenges of dark web access. While several tools for crawling the dark web have been developed in recent years, such as TorBot [4] and CRATOR [5], they are typically designed as traditional on-premise solutions. This architectural choice introduces significant criticalities: they inherently suffer from limited horizontal scalability and struggle to dynamically adapt to sudden workload spikes or the frequent connection timeouts that are typical of the unstable Tor network. Furthermore, scaling these traditional tools requires costly hardware provisioning and complex manual coordination of distributed agents. The first goal is to take advantage of the benefits offered by cloud-native infrastructures, such as elasticity, auto-scaling, and the resilience

of serverless functions, to address the complexity associated with collecting and analyzing content on the dark web.

The ultimate goal is therefore to **design and deploy** a **serverless architecture for dark-web crawling** that is both lightweight, distributed, and anonymous for the Tor network's automated exploration, highlighting its potential benefits and analyzing its operational aspects. Since this kind of work is pretty new in this context, the main research question that we want to answer is:

RQ$_1$. *Is it possible to design and develop a distributed serverless architecture for crawling the dark web?*

At first glance, this question may seem trivial, but it is not. In fact as we will see later in next chapters, the dark web is a very complex environment where any website cannot be reached through the standard HTTP protocol, but requires a specific protocol (Tor) to access it.

Once we have answered the main research question, we can move on to the following sub-questions that are strictly related to the main one.

RQ$_2$. *What are the main differences between the on-premise version and the serverless version of the crawler?*

The second research question aims to understand the differences between having a dark web crawler in a serverless environment and to have it in an on-premise one. For this purpose, we will consider all the advantages and disadvantages of both alternatives to see which one is best suited for this use case.

RQ$_3$. *How can we implement a tool that allows users to crawl the dark web in a open-source and distributed way?*

The third research question focuses on implementing a platform that allows users to crawl the dark web directly from their browsers, without the need to install any software.

1.3 Contributions

Given the research questions that will guide the research work, the contributions that this thesis will show in the following chapters can be summarized as follows:

1. **Design and development** of a distributed serverless architecture for dark web crawling.
2. **Comparison and differences** between the on-premise version of different dark web crawlers and the serverless architecture.
3. **Development** of a cloud-based web-application for any user who wants to crawl and see the results.

1.4 Results

The research carried out proved the technical feasibility of a distributed serverless architecture for monitoring the dark web, showing that it's possible to get around the inherent issues with the Tor network, like high latency and the instability of Onion services. The developed system has proven to be able to effectively manage automated exploration at multiple levels of depth, while ensuring maximum security and anonymity for agents through the use of containerized Tor proxies and periodic circuit rotation mechanisms.

One of the most significant outcomes of the work is the comparative analysis with traditional on-premise solutions. Although the traditional solution maintain an advantage in terms of pure speed, the serverless approach stood out for its superior robustness and, in particular, its exceptional horizontal scalability, which allows download times to be optimized as the number of workers employed increases.

From an economic point of view, the study highlighted how Microsoft Azure's pay-per-use model is extremely advantageous for intermittent workloads or those typical of academic research, eliminating the need for initial investments in hardware. Finally, the creation of a web interface accessible via browser confirmed the possibility of making these technologies usable in cloud-native mode, drastically simplifying the start-up and monitoring of crawling operations without requiring local installations.

1.5 Thesis Structure

This thesis is structured as follows:

- **Chapter 2 - Theoretical Background**: discusses the foundational concepts necessary to understand the Dark Web, the Tor network, and the paradigms of Cloud Computing and Serverless architectures.
- **Chapter 3 - Related Works**: provides a comprehensive review of the state of the art in web-crawling, with a focus on existing distributed and cloud-based solutions.
- **Chapter 4 - Design and Implementation of Serverless CRATOR**: describes the architectural design and the technical implementation of the proposed crawler, detailing how the components interact to achieve a scalable serverless environment.
- **Chapter 5 - Empirical Evaluation of Serverless CRATOR**: outlines the experimental protocol, the hardware and software configurations used for testing, and the data analysis methodology employed to map metrics to the research questions.
- **Chapter 6 - Results and Discussion**: presents the experimental findings organized by Research Question, providing a comparative analysis between the on-premise and serverless deployment models.
- **Chapter 7 - Conclusions**: summarizes the outcomes of the study, discusses its limitations, and suggests possible directions for future research.

CHAPTER 2

Theoretical Background

2.1 The World Wide Web

The World Wide Web, commonly referred to as the *web*, is a **distributed hypertext information system** which is accessible via the Internet [6].
Designed by **Tim Berners-Lee** in 1989 at CERN (Conseil Européen pour la Recherche Nucléaire) in Geneva, its initial objective was to facilitate the **exchange** and **updating** of information between research groups [1]. Today it represents much more than its original goal, in fact, thanks to its intentionally scalable and highly adptive structure, it now contains billions and billions of pages.
Unlike the Internet, which constitutes the underlying network infrastructure, the web represents a service that operates on it, allowing users to easily access documents and multimedia resources through a user interface, tipically a **web-browser**.
The key concept behind the web is the **hypertext**, a term coined by **Ted Nelson** in 1965 and defined as *"a set of documents interconnected by links that allow a non-linear navigation of the information space."* [7].

2.1.1 Hypertextual Structure

The web is structured as a **global hypertext system**, in fact every resource available online is **uniquely identified** and can contain links to other resources. As a theoretical matter, the entire web can be modelled as a **directed graph** where the nodes represent the documents and the edges are the hyperlinks between them. [8]. From the user's point of view, this type of structure allows easier and more intuitive browsing, making it simple to switch between related contents. From a computational perspective, this complex structure introduces some issues related to crawling coverage, the discovery of content that is not easily accessible, and the efficient handling of redundant or cyclic links.
Furthermore, there are important implications from the viewpoint of **indexing**, **navigability** and **computational analysis** of the web. Search engines such as **Google** [9] base their functioning exactly on algorithms that analyse the web graph to determine the importance of pages, such as the well-known **PageRank** [10].

2.1.2 Fundamental Web Technologies

The functioning of the web is based on **three fundamental technologies**, which define how information is identified, transmitted and presented.

Uniform Resource Identifier (URI)

An URI is a **unique identifier** that allows any resource available on the web to be recognised. It can describe either:

- a **location**, in which case we speak of an URL (**Unifor Resource Locator**).
- a **name**, i.e. a URN (**Uniform Resource Name**).

In the context of the web, URLs are commonly used to identify and access online resources. An URL is structured as follows:

```
<scheme>://<domain>:<port>/<path>?<querystring>#<fragmentid>
```

where each component has a specific role:

- `<scheme>`: specifies the protocol used to access the resource (e.g. http, https, ftp, etc...).
- `<domain>`: represents the domain name of the server on which the resource is located (e.g. www.example.com).
- `<port>`: specifies the communication port to which the request is sent (e.g. 80 for HTTP and 443 for HTTPS). It is optional and often omitted when using the default port.
- `<path>`: describes the route on the server machine to get to the requested resource (e.g. /products/list.html).
- `<querystring>`: an optional part that allows parameters to be passed to the server (e.g. ?category=books&color=red). It is often used to filter, sort or customise results.
- `<fragmentid>`: also optional, is identified by the symbol # and allows reference to a specific section within the resource itself.

HyperText Transfer Protocol (HTTP)

HTTP is the standard protocol used for communication between client and server in the context of the World Wide Web. It works using a **client-server model**: the client, generally a browser, sends a request to the server, which replies with the requested information [11].
Unlike other application layer protocols, such as FTP, HTTP is **designed to be stateless**, i.e. without maintaining state between requests. This means that once a request (or a small group of related requests) has been completed, the connection between client and server is normally closed. This behaviour is particularly convenient for the modern web, where pages may contain content from different servers. Limiting the number of active connections to only those that are strictly necessary improves efficiency on both the client and server side.

The http protocol works as follows:

1. **Connection**: the client establishes a connection with the server, generally using the TCP protocol on port 80 for HTTP or port 443 for HTTPS.

2. **Request**: the client sends an HTTP request that includes:

   - **HTTP method**: specifies the action to be performed (e.g. GET, POST, PUT, DELETE).
   - **URL**: specifies the requested resource.
   - **Protocol Version**: e.g. HTTP/1.1.
   - **Header**: contains metadata such as `User-Agent`, `Accept`, `Host`.
   - **Body**: present in some methods (e.g. POST), it contains data to be sent to the server.

3. **Processing**: the server receives and processes the request, then performs the necessary operations to generate a response.

4. **Response**: the server sends an HTTP response that includes:

   - **Status Line**: includes the protocol version, the status code (e.g. `200 OK`, `404 Not Found`) and a textual description.
   - **Header**: provides information such as `Content-Type`, `Content-Length`, `Set-Cookie`, etc. .
   - **Body**: includes the required data, which may be an HTML page, a media file, JSON data, etc. .

5. **Connection Closure**: depending on the protocol version and settings (e.g. use of the Connection: `keep-alive` header), the connection may be closed immediately after the response or kept active for further requests.

Figure **2.1** [12] shows an example of a typical HTTP request and response.

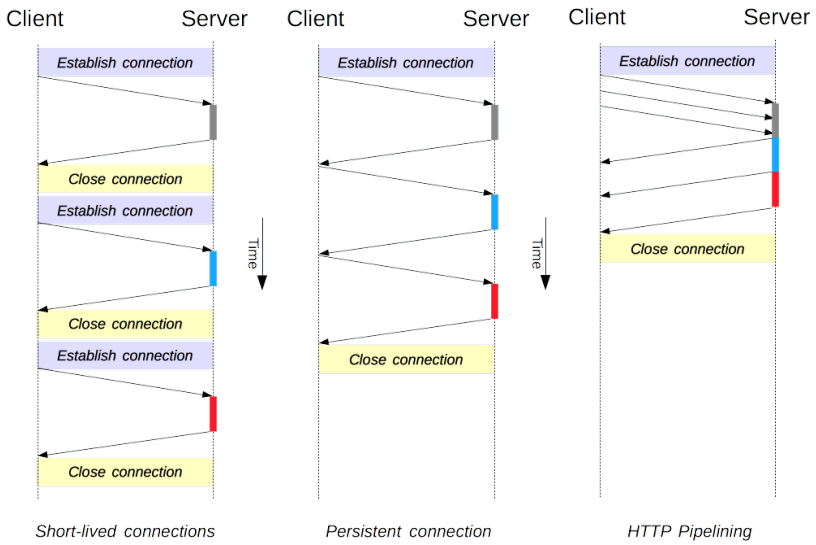

Figure 2.1: Example of HTTP request and response, with different connections.

HyperText Markup Language (HTML)

HTML is the **markup language** used to define the logical structure and content of web pages. Through **HTML tags**, it is possible to organise text, images, links, forms and other elements, making it possible to construct interactive documents that can be read by both humans and automatic agents, such as crawlers and scrapers.

2.2 Web Layers: Visibility, Access and Anonymity

The complex and highly adaptable nature of the web has led to a layering of content, consisting of different sections that vary in accessibility, visibility and level of anonymity. These sections are commonly divided into three main categories: the **Surface Web**, the **Deep Web** and the **Dark Web**.

2.2.1 The Indexed Web: surface-level content

The Surface Web, also known as the *Indexed Web* or even *Lightnet* [13], represents the portion of the web that is **publicly accessible** and **indexed** by traditional search engines such as Google or Bing [14]. It includes public websites, blogs, e-commerce platforms, social media and educational resources. Consumers can access this content without the use of special software or credentials.

The exact number of indexed pages on the web is not publicly known, as search engines do not make such detailed data available. Furthermore, it is a dynamic value that varies continuously: new pages are constantly added to the index, while obsolete ones are removed. According to some estimates, the number of indexed pages is between 50 and 200 billion [15]. Despite representing a huge amount, this number is only a small fraction of the entire content on the web.

2.2.2 The Hidden Web: non-indexed and restricted content

The Deep Web, also known as the *Hidden Web*, covers all those parts of the web that are **not indexed** by traditional search engines [16]. Resources on the deep web are accessed in a similar way to the surface web, but may **require authentication** or the adoption of **additional security measures**. This category includes many everyday services, such as e-mail, online banking, private or academic databases and streaming platforms.

The difficulty in indexing these types of content comes from the inability of web crawlers to overcome automatic authentication systems, such as CAPTCHAs, or to use login credentials required to view certain resources [17].

Unlike the surface web, the deep web is a significant part of the overall web, estimated at up to **90%** [2]. Content on the deep web can be in any field and is subject to different regulations and security levels.

2.2.3 The Anonymous Web: encrypted and decentralized networks

The term Dark Web refers to all those contents of the deep web in darknets, which can only be reached via specific software, authorisations and configurations. [18].

The technology behind the dark web is the **Darknet** [19], i.e. a virtual private network to which only self-identified and trusted users can connect. All darknets need specific software or particular network configurations to be used, such as the **Tor Network** [20]. In addition, darknets guarantee the anonymity of users by routing traffic through a network of voluntary nodes and encrypting data in multiple layers.

Content on the dark web ranges from legitimate resources to illegal and dangerous material [21]. Among the illegal content we can find:

- **Black Market**: clandestine e-commerce platforms where drugs, weapons, fake documents, stolen data and other illegal goods are traded. Known examples include Silk Road and AlphaBay.
- **Illegal Pornography**: child pornography material, often associated with extreme forms such as hurtcore, involving violent sexual abuse of minors.
- **Criminal Services**: hacking offers, phishing, distribution of malware, ransomware-as-a-service and botnets.
- **Financial Fraud**: sale of stolen bank data, cloned credit cards, money laundering services via cryptocurrency and scam schemes.
- **Extreme Content**: violent material such as torture, animal cruelty and *"red rooms"*, so-called virtual rooms where acts of violence are broadcast live.

Among the legal ones we can also find alternative **social networks**, **forums**, **media news**, **whistleblowing platforms**.

2.3 Onion Routing and the Tor Network

Web communication requires not only protecting the content of messages, but also preserving the identity of interlocutors [22]. Although traditional encryption protects data from eavesdropping, it does not prevent traffic analysis, which can reveal who is communicating with whom and how often. To address these issues, **onion routing**, a technique designed to ensure anonymity and security in online communications, was developed.

2.3.1 What is onion routing

Onion routing is a technique for **anonymous communication** on a computer network [23]. On an onion network, messages are encapsulated in **consecutive layers** of encryption, as shown in figure **2.2**, and are transmitted through a series of nodes called **onion routers** [22]. Each node removes one layer of encryption to reveal the next destination node, without knowing the origin or final destination of the message.

This process ensures that no intermediate node can determine both the sender and the final receiver, thus preserving the anonymity of the communication.

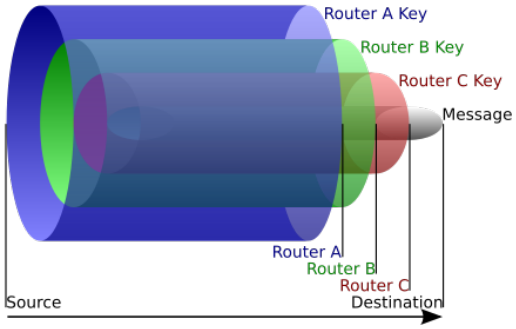

Figure 2.2: Message encapsulated in multiple layers of encryption.

2.3.2 Functioning of onion routing

The process of data transmission through onion routing involves the following steps [24]:

1. **Circuit selection**: the client that wishes to communicate selects a route through the network, consisting of a sequence of nodes and called *circuit* or *chain*, one of which is the entry node, than several intermediate nodes and at the end the exit node.

2. **Onion construction**: the client encapsulates the original message in multiple layers of encryption, one for each node in the path. Each layer contains the information needed to route the message to the next node and a key to decrypt the next layer.

3. **Onion Transmission**: the encrypted message is sent to the input node, which uses its key to remove the first layer of encryption, revealing the address of the next node. This process continues to the output node.

4. **Delivery to Final Recipient**: the output node removes the last layer of encryption and sends the decrypted message to the final recipient.

5. **Recipient's Response**: the recipient's response follows the same path backwards. Each node adds an encryption layer, and the final client, knowing all the keys, can fully decrypt the received reply.

This mechanism ensures that each node only knows the previous and next nodes, preventing complete path reconstruction and preserving the anonymity of users.

2.3.3 Vulnerabilities of onion routing

Although onion routing is an advanced technology, is not without its vulnerabilities. Some attacks, in fact, can **compromise the confidentiality** of users, especially if conducted by attackers with observation capabilities at multiple points in the network.

One of the main threats at the protocol level is **traffic correlation attacks** [23]. These attacks are based on the **simultaneous observation of traffic** entering and leaving the network. An attacker capable of monitoring both the ingress and exit nodes may be able to correlate the flow of packets, analysing the timing, size and volume of traffic. Even if the data is encrypted within the network, these metadata features can be used to infer the identity of the user or the websites visited, thus compromising the anonymity guaranteed by the system.

A second vulnerability concerns the **exit nodes** [23], which represent the end point of the onion network from which traffic is routed to the open Internet. At this stage, if the connection to the destination site is not protected by **end-to-end encryption**, as is the case of HTTPS, data leaves the network in an unencrypted format. A compromised exit node or one managed by a malicious actor can then intercept, record or manipulate the information in transit. This may include login credentials, message contents, personal data and other sensitive information transmitted unencrypted.

2.3.4 The Tor Network

Tor, which stands for *The Onion Routing*, is a **low-latency anonymous communication system** that represents the second generation of onion routing [25]. Unlike the first implementation of onion routing, it introduces significant improvements in

terms of security, scalability and usability, making it one of the most popular tools for anonymous online browsing.

2.3.5 Improvements over the old Onion Routing design

The improvements introduced by this new communication system address some of the problems present in the original system, introducing new tools and mechanisms to improve both the anonymity and security of data transmitted over the network [25]. Among these, the main improvements are:

- **Perfect Forward Secrecy**: circuits are created using temporary keys. This implies that even if a private key is compromised, past traffic cannot be decrypted retroactively, as each session uses unique ephemeral keys.
- **Shared circuits**: multiple TCP flows can share the same circuit, improving network efficiency and reducing computational load. It also helps mitigate the risk of traffic correlation by increasing user anonymity.
- **Congestion control**: a decentralised control mechanism based on end-to-end ack allows edge nodes to detect congestion on the network and dynamically adapting data transmission rates, improving overall performance and reducing latency.
- **Directory servers**: Tor uses a set of directory servers managed by trusted entities, known as *Directory Authorities*, to maintain a consistent and up-to-date view of the network. These servers provide clients with reliable information on available nodes, improving the security and stability of the network.
- **End-to-end integrity checking**: the tor network performs integrity checks on data before it leaves the network. This ensures that the data has not been altered during transit, protecting the user from potential man-in-the-middle attacks.

2.3.6 Tor network architecture

The Tor network architecture is designed not only to ensure anonymity and security of online communications, but also to fulfil fundamental criteria such as

usability, **flexibility** and ease of **deployment** [25]. These aspects are essential to ensure that Tor is accessible to a wide range of users and can be deployed effectively in different contexts.

There are primarily three components that are part of the network:

- **Tor Client**: software used by users to connect to the network. It builds circuits across the network by randomly selecting a set of nodes and negotiating session keys with each of them.
- **Relay**: voluntary servers that form the infrastructure of the Tor network, identified by a public key and registered with *Directory Authorities*. They fall into three main categories:
  - **Guard Relay**, the first node of the circuit, carefully selected to limit the exposure of the user's IP.
  - **Middle Relay**, intermediate node that forwards traffic without knowing either the source or the destination.
  - **Exit Relay**, last node in the circuit, from which traffic exits to the open Internet.
- **Directory Authorities**: trusted servers that maintain a consistent and up-to-date view of the state of the network.

The Tor client builds **three-hop** cryptographic circuits [26], negotiating session keys with each node via a Diffie-Hellman based protocol. Each node decrypts only its respective layer, ensuring that no relay knows both the source and destination of the traffic. In addition, circuits are **periodically rebuilt** to improve security and anonymity. In particular scenarios where anonymity is not a priority, Tor allows the construction of **single-node circuits**. In such cases, the client establishes a direct connection with a single relay, reducing latency but giving up complete anonymity.

2.4 Web Crawling

The layered structure of the web and the huge amount of heterogeneous content within it make the acquisition of information complex [27]. In addition, the lack of a

uniform content structure makes navigation and exploration even more complicated. In this scenario, automated tools such as **Web Crawlers**, which facilitate efficient search and data extraction, play a key role.

2.4.1 Functioning

A web crawler is **automatic software** designed to systematically and accurately traverse the web [28]. Also known as *spiders*, *robots* or *worms*, crawlers have the main task of building a navigation graph from a root URL, identifying and following links on pages to continue exploration in a recursive manner.

It consists of three main components [27]: a **frontier**, which collects and manages the URLs to be visited; a **page downloader**, responsible for downloading web pages; and a **web repository**, which receives the downloaded pages from the crawler and stores them in a database.

The functioning of a crawler can be described in the following steps [28] and it's illustrated in figure **2.3**:

1. **Extraction** of an URL from the frontier from which to start the crawling process.
2. **Connection** to the selected URL and **download** of the corresponding web page.
3. **Analysis of content** of the page to identify new links, which are placed back into the frontier for further exploration.

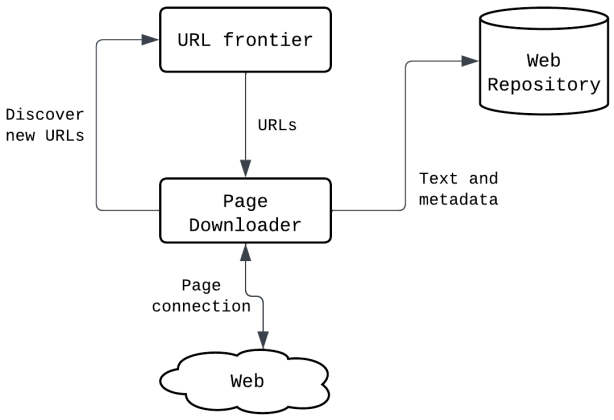

Figure 2.3: Web crawler architecture.

2.4.2 Crawling policies

The performance of a web crawler is governed by a set of policies that define its mode of operation and optimise its execution [29]:

- **Selection Policy**: it determines which pages should be downloaded and which should be ignored. A crawler does not download all the content but only selects the ones considered relevant. In order to do this, metrics are used to evaluate the relevance of pages, based on criteria such as *quality* and *popularity*.
- **Re-Visit Policy**: it determines how often a page that has already been visited should be re-examined for updates. The principal metrics involved are:
  - **freshness**, which indicates whether, at a given time t, the locally saved copy is still the same as the online version.
  - **age**, which measures how much time has passed since the last synchronisation, giving an indication of how outdated the local copy is.
- **Politeness Policy**: it defines how the crawler should behave to avoid overloading the servers of the visited websites. Since the crawler simulates the navigation of a real user, without adequate control it may generate an excessive number of requests, causing *slowdowns* or *inefficiencies*.
- **Parallelization Policy**: it controls the coordination between multiple distributed crawlers operating in parallel. The goal is to reduce overhead from concurrent processes and maximise download efficiency by avoiding *duplication* and *conflicts* between crawlers.

2.4.3 Crawling strategies

The effectiveness of a crawler depends not only on its internal policies but also on the specific **crawling strategies** adopted. These strategies can be categorized into two levels: the fundamental **graph traversal algorithms**, which dictate the order of node visitation, and **operational approaches**, which are tailored to specific goals in order to meet particular needs [27].

Graph Traversal Algorithms

At its core, a crawler treats the web as a directed graph. The order in which URLs are extracted from the frontier typically follows one of two classic methods:

- **Breadth-first search (BFS)**: the crawler visits the nodes closest to the root of the tree first, and then moves on to the nodes further away.
- **Depth-first search (DFS)**: the crawler visits the nodes farthest from the root of the tree first, and then moves on to the nodes closest to it.

These two methods differ in the order in which they visit nodes, affecting the way URLs are extracted from the frontier as well.

Operational crawling strategies

Beyond basic traversal, several specialized strategies exist to meet specific technical or thematic needs:

- **General Purpose Crawling**: a basic strategy that aims to download **as many web pages as possible**, starting from one or more initial URLs provided by the user, without any thematic restriction.
- **Focused Crawling** (or *Topic Crawling*): in this approach, the crawler focuses exclusively on pages that are relevant to a given **topic**. This reduces network traffic, the number of unnecessary downloads and the amount of computing resources consumed.
- **Incremental Crawling**: the crawler **periodically updates** already downloaded pages, replacing obsolete ones with newer versions. This is made possible by re-visit policies that guarantee high freshness of content over time.
- **Distributed Crawling**: involves the use of **several distributed crawlers**, each operating on a distinct portion of the web. The activity is coordinated by a central server, which assigns tasks and manages synchronisation.
- **Parallel Crawling**: in this strategy, **several processes work in parallel**, simultaneously performing download operations and discovering new links. This approach allows a significant increase in overall system performance.

2.5 Cloud Computing

The notion of cloud computing traces its roots back to the 1960s, when people started talking about **utility computing** [30, 31], i.e. the idea of providing computational capacity as a service, similar to household utilities such as electricity or water. Although at the time this idea was still far from practical realisation, it represented the basis from which the cloud would later take shape.

As technology has progressed, especially since the 2000s, cloud computing has assumed an increasingly central role in the world of IT and digital service delivery. The spread of **high-speed Internet connections**, **virtualization**, and the availability of **large data center infrastructures** have made it possible to deliver computing resources remotely, on demand and in a highly scalable manner. As a result, cloud computing has become a fundamental part of modern IT industries, and the interest in this technology has grown exponentially over the years, as figure **2.4** shows [32].

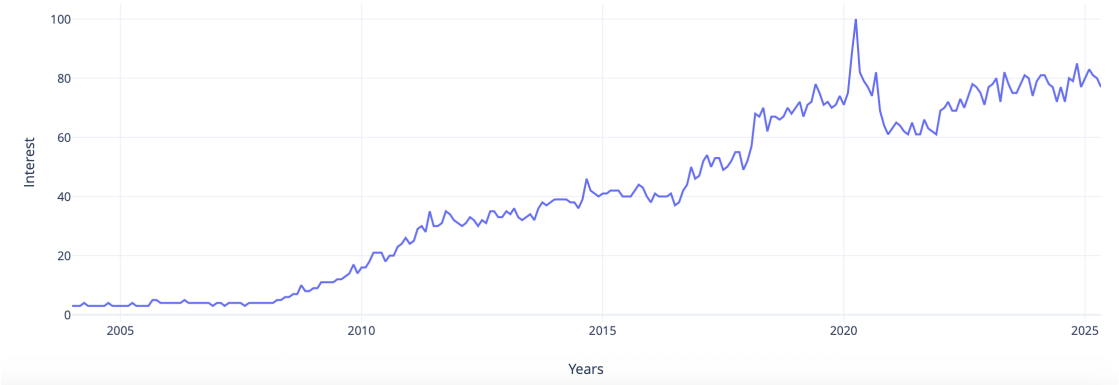

Figure 2.4: Interest over time for "Cloud Computing" searches on Google, from 2004 to 2025.

There is no single definition of cloud computing. Some intend it in a broad sense, including both **hardware resources** (such as servers, storage and networking) and **software applications and services** delivered over the Internet [33]. Others describe it as a distributed computing paradigm in which IT resources such as computing power, storage space, or development platforms, are provided on-demand, often through a consumption model [34].

However, among the various definitions proposed over time, the most recognised and authoritative was provided in 2009 by the **National Institute of Standards and**

Technology (NIST) [35], which describes cloud computing as: *"A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction."*

The last definition highlights some key properties [36, p. 58-62] that distinguish the cloud:

- **On-demand self-service**: users can access and manage resources autonomously, without the need to interact with the service provider.
- **Ubiquitous access**: resources are available through a network, typically the Internet, and can be used by any connected device.
- **Multitenancy e Resource Pooling**: physical resources are shared between multiple users, allowing for more efficient use of infrastructure and reduced costs.
- **Rapid elasticity**: resources can be scaled rapidly according to demand, allowing users to flexibly increase or decrease capacity.
- **Measured service**: resource consumption is monitored and metered, allowing users to pay only for what they actually use.
- **Resilience and availability**: cloud architectures are designed to ensure high availability and fault tolerance, reducing the risk of downtime and data loss.

2.5.1 Cloud enabling technologies

The diffusion and establishment of cloud computing has been made possible by a number of **enabling technologies** that have revolutionized the way IT resources are delivered and managed [36, p. 30-32]. Among these technologies, the most significant are without doubt: **virtualization**, i.e. the creation of virtual instances for IT resources; the emergence of technologies such as **clustering**, **grid computing** and **peer-to-peer**; the creation of distributed and scalable data centers; and the adoption of **high-speed networking** technologies.

2.5.2 Cloud Delivery Models

Cloud services can be deployed according to different service models, each of which provides a **different level of abstraction and control** over resources [36]:

- **Infrastructure-as-a-Service (IaaS)**: it provides virtualized hardware resources, such as servers, storage, and networking, allowing users to manage and configure their own applications and operating systems. This model offers the **highest level of control and flexibility**, but also requires **greater responsibility** for managing resources.
- **Platform-as-a-Service (PaaS)**: it provides a complete development environment, including tools and services for creating, deploying, and managing applications without having to worry about the underlying infrastructure.
- **Software-as-a-Service (SaaS)**: it provides software applications accessible via the Internet, eliminating the need for local installation and management. Users can access applications through a Web browser, paying only for actual use.
- **Function-as-a-Service (FaaS)**: it permits the execution of functions or microservices in response to events, without the need to manage the underlying infrastructure. This model is the basis of **serverless computing**.

Figure **2.5** gives a visual representation of the different delivery models and their respective levels of abstraction.

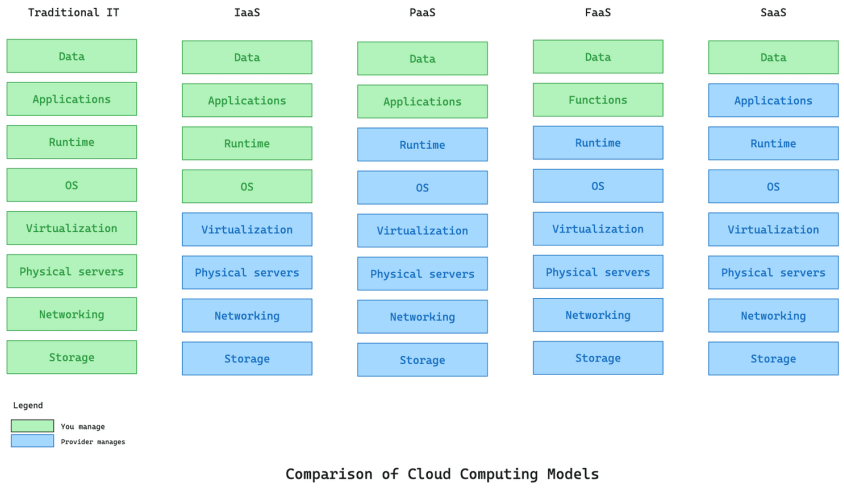

Figure 2.5: Cloud delivery models differences.

2.5.3 Cloud Deployment Models

A cloud environment can be deployed according to three main deployment models, which differ in the degree of accessibility and control of resources [33, 36]:

- **Public Cloud**: resources are available to the public. Users can access the resources through the Internet in a pay-per-use manner.
- **Private Cloud**: resources are dedicated to a single client or organization and can be managed internally or by an external vendor.
- **Hybrid Cloud**: it combines elements of public and private clouds, enabling organizations to take advantage of both models.

2.6 Serverless Computing

Serverless computing, also referred to as **Function-as-a-Service**, is *"an application development and execution model that enables developers to build and run application code without provisioning or managing servers or back-end infrastructure."* [37]. On the other side, the cloud provider takes care of the management and automatic scaling of the resources needed to run the code.

2.6.1 History and Evolution

The appearance of serverless in the cloud computing landscape goes back to 2014 when **Amazon Web Services (AWS)** launched the **AWS Lambda service** [38]. Since then, many other cloud service providers have followed the example, offering similar solutions, such as **Google Cloud Functions**, **Microsoft Azure Functions**, and **IBM Cloud Functions** [39]. As already seen for the cloud computing growth, the interest in serverless computing has also grown significantly over the years since it's first appearance, as shown in figure **2.6** [32].

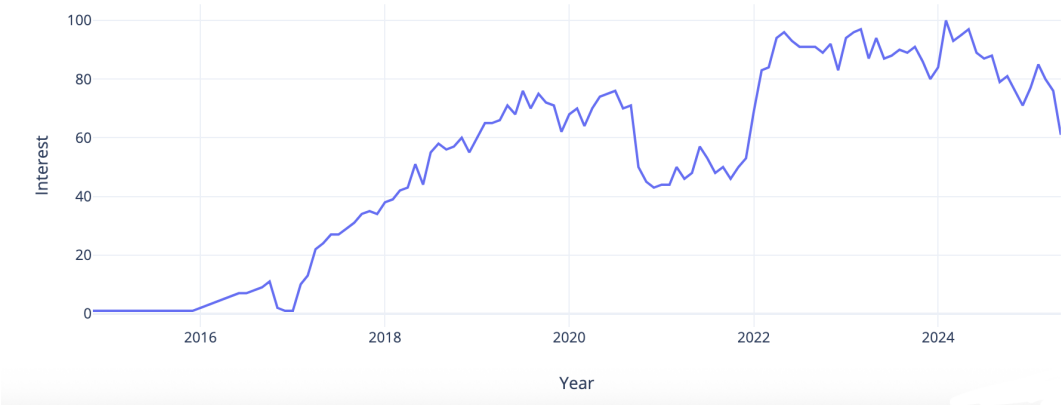

Figure 2.6: Interest over time for "Serverless Computing" searches on Google, from 2014 to 2025.

The serverless model is based on an *event-driven architecture*, in which code execution is **triggered by events** generated by external applications, services, or systems [40]. This paradigm enables the design of *loosely coupled applications*, marked by **independent** and **autonomous components**, thus facilitating system scalability and maintainability. The underlying infrastructure is **fully managed by the cloud provider**, relieving the developer of operational tasks such as resource provisioning, load balancing, and server maintenance.

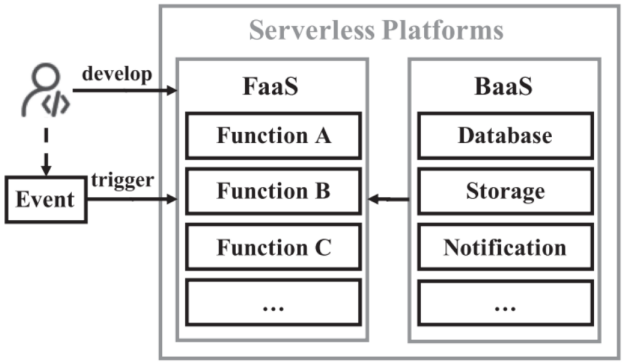

Figure 2.7: Serverless event-driven architecture.

2.6.2 Advantages and Disadvantages

There are several advantages to using serverless computing [40], including:

- **Automated scalability**: the cloud provider automatically manages resource scalability based on demand, allowing developers to focus on code.
- **Reduced Costs**: users pay only for actual code execution time, without incurring fixed costs for idle servers.
- **Fast development time**: developers can focus on writing code and implementing features, without having to worry about infrastructure management.

On the other hand, there are also some drawbacks and limitations [39, 40]

- **Execution constraints**: cloud service providers impose time and resource limits on function execution, which can be problematic for long-term or compute-intensive applications.
- **Vendor lock-in**: using a specific provider can make it difficult to switch to another provider or a traditional architecture.
- **Complex debugging and monitoring**: debugging and monitoring of serverless functions can be more complex than in traditional applications because the code is executed in a distributed and abstracted environment.

- **Latency issues**: the overhead associated with starting serverless functions can introduce latency, especially for functions that are not executed frequently.
- **Efficiency**: some aggregation and scraping functions may take longer to execute in a serverless environment than in a traditional architecture, due to the latency introduced by the creation and destruction of instances.
- **Fine-grained coordination**: the creation of complex applications requires fine-grained coordination among various functions, which can make it difficult to manage and monitor the entire system.

CHAPTER 3

Related Works

There are numerous studies in the field of web crawling that focus both on designing progressively more efficient **crawling strategies** and on extracting data by creating increasingly high-performance **algorithms** and **architectures**.

This chapter will present some of the most significant works in this field, paying attention to works that are in the area of **distributed** and **serverless** crawling.

3.1 Traditional web crawling

Baeza-Yates *et al.* (2005) [41] conducted a comparative study between different crawling strategies, comparing approaches without extra information with others that exploit **historical data** obtained from previous crawling sessions. The results showed that strategies based on historical information, computed with different strategies and different pagerank algorithms, perform better than traditional strategies.

In addition to traditional strategies, other alternative techniques have been explored, such as **Genetic Algorithms** or **Bayesian Classifiers** for selecting pages to visit. Chen *et al.* (1998) [42] conducted a comparative study between a traditional crawler, based on a BFS strategy, and one based on a genetic algorithm. Although the results showed no significant differences in overall performance, the genetic crawler

demonstrated a **greater ability** to discover new pages due to the algorithm's own mutation and crossover mechanisms.

Wang *et al.* (2010) [43] developed a focused crawler that uses a Bayesian classifier to select which pages to visit. With this approach, the crawler is able to select **the most relevant pages** for the domain of interest, while discarding the less relevant ones. The results showed the effectiveness of the proposed architecture, pointing to improved performance compared to traditional crawlers.

3.2 Distributed web crawling

Boldi *et al.* (2004) [44] presented a distributed crawler called **UbiCrawler** designed to be highly scalable and capable of handling large volumes of data. This crawler uses a distributed architecture in which each *agent* performs **breadth-first scanning** from a seed host. Each agent is responsible for crawling a **subset of URLs**, this to prevent work duplication and to comply with politeness policies. Although the researchers found some difficulties related to the programming environment, particularly the need to rewrite and adapt some default Java classes to make them compatible with a concurrent and distributed context, the crawler still demonstrated a good ability to handle a large number of requests while maintaining decent performance in a multi-threaded environment on each agent.

Lee *et al.* (2009) [45] instead proposed a **single-server crawler** called **IRLbot**. The researchers, after a deep analysis on the problems related to other established crawlers in the context of distributed crawling, designed a crawler capable of handling a very large amount of data and pages. Thanks to three innovative technologies, **DRUM (Disk Repository with Update Management)** for handling data, **STAR (Spam Tracking and Avoidance through Reputation)** for handling spam and information-intensive domains, and **BEAST (Budget Enforcement with Anti-Spam Tactics)** for handling URL queues, the crawler showed significant results in terms of performance and downloaded pages (about **6 billion** pages downloaded in 42 days of crawling) in relation to other distributed crawlers. This showed that a distributed approach is not necessary to achieve high performance, but that similar results can be achieved even with a single-server crawler.

3.3 Serverless web crawling

Serverless is a relatively new approach in the field of web crawling, in fact there are few studies that have focused on this topic. This is mainly due to the fact that serverless architectures are still in the early stages of development and there are still many challenges to be addressed, like the **exectuion time limit** of the functions, the **cold start** problem, and the need to **manage state** across multiple invocations.

However, some researchers have begun to explore the potential of serverless architectures for web crawling.

For example, Stevenson (2021) [46] proposed a serverless web crawler that leverages **AWS Lambda** and **AWS Step Functions** to create a scalable and cost-effective crawling solution.

Martinaitis (2020) [47] conducted a comparative study showing that serverless architectures are **more cost-effective** than traditional virtual machines for web crawling tasks. In his post, he also highlighted the advantages of serverless architectures, such as automatic scaling and reduced operational costs.

Despite the lack of extensive research in this area so far, the potential of serverless architectures for web crawling is promising. The ability to scale automatically and the reduced operational costs make serverless architectures an attractive option for web crawling tasks, especially for large-scale crawlers that need to handle a high volume of requests. The basic architecture of a serverless web crawler of this type is shown in figure **3.1**.

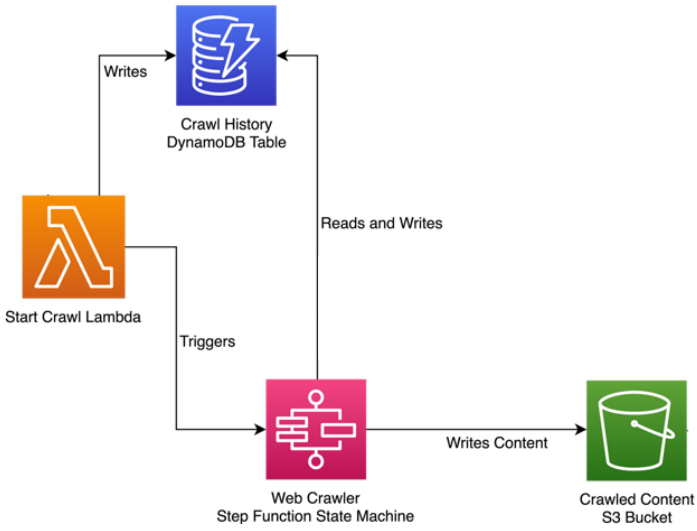

Figure 3.1: Serverless web crawler architecture.

3.4 Dark Web crawling

Crawling the dark web presents **unique challenges** and requires **specialized techniques** compared to traditional web crawling. A crawler designed for the dark web must be able to handle the specific characteristics of this part of the internet, such as the use of Tor and other anonymizing networks, as well as the presence of hidden services and dynamic content.

Narayanan *et al.* (2020) [4] proposed an OSINT tool called **TorBot**, which is a crawler designed to crawl the dark web and collect data from hidden services. The main feature of this crawler is its ability to handle the specific characteristics of the dark web, such as the use of Tor and other anonymizing networks.

Bergman *et al.* (2023) [48] conducted a systematic literature review on dark web web crawling, identifying a total of 34 web crawling tools. Additionally, they proposed a new crawler starting from the analysis of the existing ones and they got some interesting results both on the clear web and on the dark web.

De Pascale *et al.* (2024) [5] proposed a novel dark web crawler named **CRATOR**, specifically designed to overcome several limitations of existing crawling solutions. The authors developed a tool capable of addressing most of the challenges associated with dark web crawling, such as cookie rotation, CAPTCHA solving for authen-

tication, anonymous access to hidden services, and efficient management of the crawling process. Experimental results demonstrated that CRATOR outperforms existing crawlers in both performance and data collection capabilities.

This last work represents the starting point of this thesis, as it provides a solid foundation for all the tests and experiments conducted in the following chapters.

CHAPTER 4

Design and Implementation of Serverless CRATOR

4.1 Introduction

This chapter presents the methodology adopted for the design and implementation of the serverless web crawler targeting the dark web, with specific focus on `.onion` services accessible via the Tor network.

The goal is to detail the **architectural decisions**, **component interactions**, **data flow**, and some technical strategies adopted to ensure anonymity, scalability, and modularity. The system was designed following a serverless paradigm using cloud-native services, aiming to minimize infrastructure overhead and maximize scalability.

Starting from the research objectives, this chapter will outline the system architecture, the crawling workflow, the implementation choices made to implement the architecture, and the measures taken to ensure anonymity and security. Finally, it will discuss how concurrency and scalability are managed within the system.

4.2 Design Principles

The crawler was designed to address several key objectives, each of which plays a crucial role in the overall functionality and performance of the system:

- **Scalability**: since the crawling process can be resource-intensive, time-consuming, and generate a large number of URLs to be processed, the architecture was designed to be scalable by using **worker functions** that can run in parallel, coordinated by a **central orchestrator**.
- **Modularity**: the architecture was designed to be modular, with each component responsible for a specific task (queueing, crawling, storage). This allows for easier maintenance and updates, as well as the ability to add new features or components in the future.
- **Efficiency**: the architecture was designed to be efficient, with a focus on minimizing resource usage and avoiding revisiting previously crawled URLs. This is achieved by using a **URL frontier** to manage the list of URLs to be crawled, and by using a **crawling history** to keep track of previously crawled URLs.
- **Anonymity**: since the crawler is designed to operate on the dark web, it is crucial to ensure that the crawler does not reveal its identity or location. This is achieved by routing all traffic through the **Tor network** and by using a **proxy server** to handle requests.

4.3 System Architecture

This sections presents first the related architectures that inspired the design of the crawler, then the proposed architecture and the design choices made to adapt it to the dark web crawling context.

4.3.1 Related Architectures

The architecture design of the crawler was made following the general one shown in other serverless architectures. As a starting point, the architecture proposed by Stevenson *et al.* (2021) [46] was taken into consideration, which is shown in figure **4.1**.

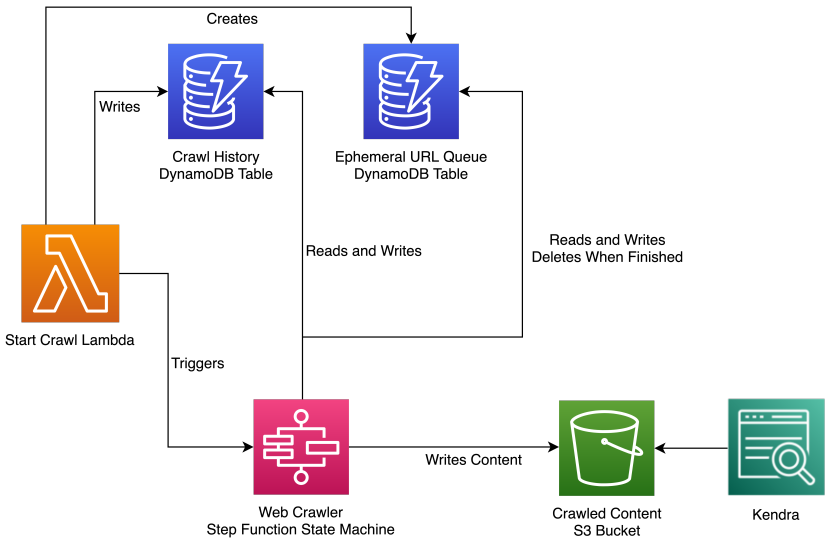

Figure 4.1: Stevenson *et al.* (2021) serverless architecture.

This infrastructure built in the AWS cloud not only includes core components such as functions (**AWS Lambda**), a data storage service (**AWS S3 Bucket**), and a NoSQL database for crawling history management (**AWS DynamoDB**), it also includes a second **ephemeral database** used for the management of the URLs frontier and a search engine (**AWS Kendra**) for browsing the downloaded data.

The successful integration of the serverless architecture with the dark web crawling requires other fundamental components. For this purpose, the solution chosen as the architecture to follow is the one proposed by De Pascale *et al.* (2024) [5], which is shown in figure **4.2**.

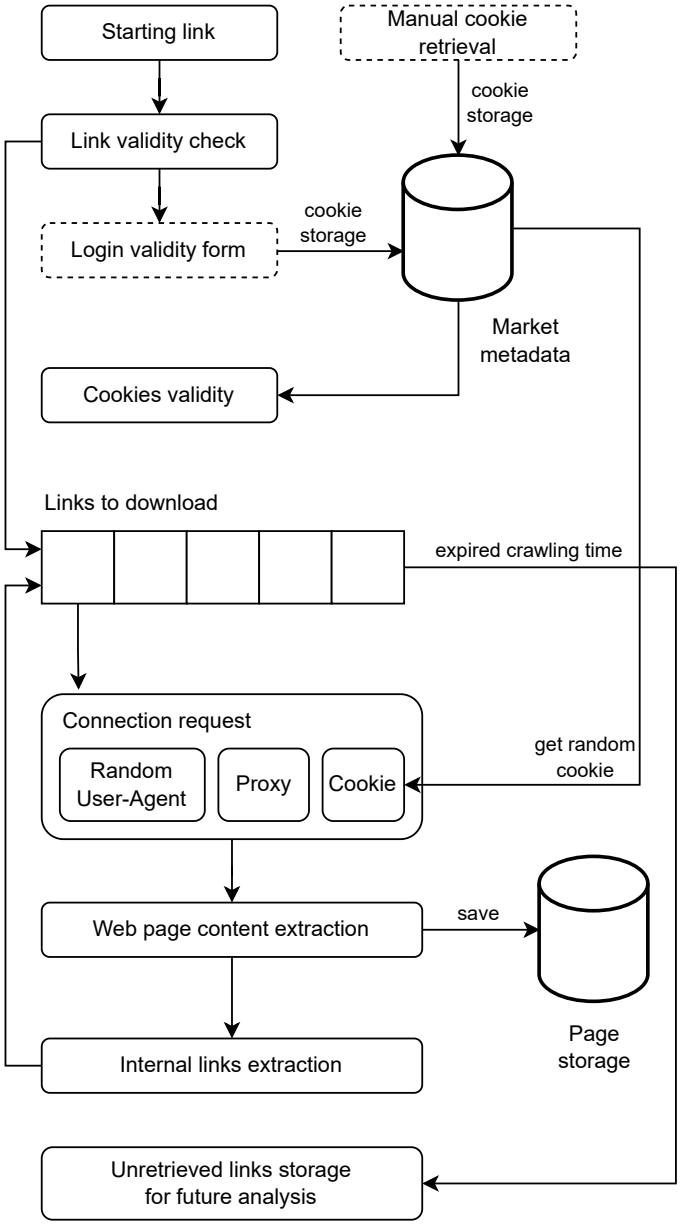

Figure 4.2: De Pascale *et al.* (2024) serverless architecture.

As shown in the figure, the architecture includes a database for managing cookies used in login forms, and requests forwarded toward the pages to visit are composed of a proxy server that is responsible for forwarding requests to the Tor network.

4.3.2 Proposed Architecture

The architecture presented in this thesis is based on the basic components of a crawling system, with some significant modifications to adapt its operation to the dark web context. In particular, three key elements are introduced:

1. a **proxy server** for outbound traffic management.
2. a **function orchestrator** for coordinating workers.
3. a **database** devoted to cookie management and storage.

The proxy server is responsible for properly **routing requests** through the Tor network, ensuring anonymity and access to `.onion` services. The orchestrator, on the other hand, is in charge of managing the workers, assigning them crawling tasks and **synchronizing** their activities. Finally, the cookie database enables the storage of **session state**, improving the efficiency and continuity of data collection operations. A high-level architecture diagram is provided in figure **4.3**.

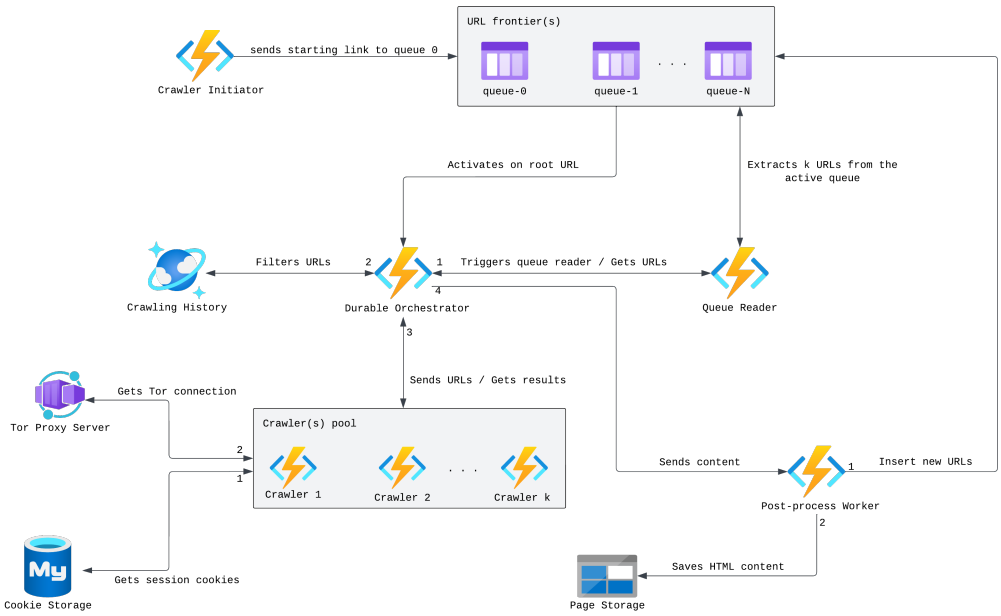

Figure 4.3: Proposed serverless architecture.

4.4 Crawling Workflow

The system follows a **multi-stage orchestration** workflow, where each stage is responsible for a specific task in the crawling process. A summary of the workflow is shown in figure **4.4**.

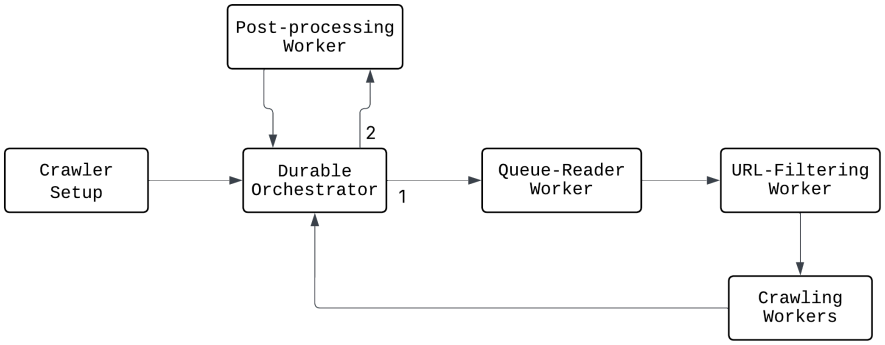

Figure 4.4: Crawling workflow overview.

As shown in the figure, the workflow consists of the following stages:

1. **Crawling Initialization**: before the crawling process begins, an utility function is executed to create the whole crawling environment. This includes creating the necessary resources such as:

   - a **storage container** for storing the crawled data.
   - a **NoSQL database** for managing the crawling history.
   - all the **BFS-level queue** for managing the URLs frontier.
   - an **utility table** for storing environment variables chosen by the user (e.g., maximum number of URLs to crawl, maximum depth, maximum number of concurrent workers).

   This function is executed **once** at the start of the crawling process, with a corresponding function executed at the end to clean up all created resources.

2. **Crawling Orchestration**: the durable orchestrator function is triggered upon the completion of the crawling initialization stage. It is responsible for managing the crawling process by coordinating the worker functions, acting like a **central controller**.

3. **Queue-Reader Worker**: the first worker function is responsible for reading and extracting the URLs from the current active queue and returning them to the orchestrator. The reader retrieves a specified number of URLs from the queue, based on the maximum number of concurrent workers configured by the user.

4. **URL-Filtering Worker**: the second worker function is responsible for filtering the URLs returned by the queue reader. It checks if the URLs are valid, e.g. not already crawled. If a URL does not pass this check, it is removed from the list of URLs to be crawled.

5. **Crawling Workers**: the third function is responsible for crawling the URLs and it is invoked by the orchestrator as many times as the number of concurrent workers allowed by the user. Each worker is assigned a URL to crawl and carries out the following tasks:

    - gets the connection to the proxy server.
    - retrieves (if needed) the cookies from the cookie database.
    - sends the request to the proxy server, which forwards it to the Tor network.
    - receives the response from the proxy server and processes it to extract the internal links.
    - sends back the crawled data to the orchestrator.

6. **Post-processing Worker**: the last worker function is responsible for post-processing the crawled data. It receives all the data obtained from the crawling workers and re-inserts the internal links into the active queue, if they are not already present in the crawling history, and saves the content of the crawled page into the storage container. It also updates the crawling history with the new URLs and their metadata (e.g., timestamp, status code, content type).

After the post-processing worker completes its task, the orchestrator resumes the loop starting from the queue-reader worker. This process continues until one of the following conditions is met:

- the maximum number of URLs to crawl has been reached.

- the maximum depth has been reached.
- there are no more URLs to crawl in any of the queues (if the current active queue is empty, the orchestrator moves to the next one).

At this point, the orchestrator function completes its execution and the crawling process is considered finished.

4.5 Implementation Choices

The implementation of the architecture was carried out using the **Microsoft Azure Cloud Platform** [49], leveraging its serverless services to create a scalable and modular crawling system. Some of the key services used in this implementation are:

- **Azure Functions**, used to implement the different components of the crawler, each one implemented as a separate function, allowing for independent scaling and parallel execution.
- **Azure Storage**, used to store the crawled data, configuration files, logs, and queues used to simulate the URL frontier.
- **Azure CosmosDB**, used to store the crawling history in a NoSQL format using the MongoDB API, the cookies and session data, allowing the crawler to maintain state across multiple requests.
- **Azure Container Apps**, used to run a containerized Tor proxy server, which is responsible for routing requests through the Tor network to access `.onion` services.
- **Azure VNet**, used to create a private network for the container apps and functions, isolating them from the public network and allowing them to communicate securely.

Some design choices were made to adapt and overcome some limitations of different azure services.

4.5.1 Azure Functions constraints

The first problem related to the implementation of the crawling system is the **execution time**. This is a critical aspect of the crawling process, as it can take a long time to crawl a large number of URLs, especially when dealing with `.onion` services that may have slow response times.
The default version of the Azure Functions service has a maximum execution time limit of **5 minutes**, which it is sufficient for some tasks. However, for the crawling process, which can take much longer, it is necessary to use the **durable functions** feature of Azure Functions. This feature allows to create stateful workflows in a serverless environment, which is particularly useful for implementing the crawling process that requires a sequence of steps that need to be executed in a specific order and can take a long time to complete.
This feature lead to the implementation of the **orchestrator function**, which is responsible for managing the crawling process by coordinating the worker functions. Thanks to the **snapshot feature** of the durable functions, the orchestrator can maintain the state of the crawling process and resume it from where it left off in case of failures or timeouts. This allows to overcome the execution time limit of the classic Azure Functions and to implement a long-running crawling process, creating different autonomous tasks, called **activity functions**, that can be executed independently without the risk of timeouts.

Application Patterns

As microsoft documentation states[1], there are different patterns that can be used to implement durable functions. In this case, the **function chaining** pattern, the **fan-out/fan-in** pattern, and the **async HTTP APIs** pattern were implemented. The first one is used to implement the **crawling process**, where each step of the process is implemented as a separate activity function, and the orchestrator function is responsible for chaining them together, as shown in code snippet **4.1**.

[1] `https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview`

```
@app.orchestration_trigger(context_name = "context")
def orchestrator_function(context: df.DurableOrchestrationContext):
        retrieved_messages = yield context.call_activity("queue_reader_function", {
            "index": queue_index,
            "max_workers": max_workers
        })

        url_to_crawl = yield context.call_activity("filter_urls", {
            "messages": retrieved_messages
        })

        # invoking crawling workers in parallel and getting results

        results = yield context.call_activity("postprocess_results", {
            "results": crawl_results,
            "depth": queue_index,
            "max_depth": max_depth
        })
```

Code 4.1: Application of function chaining pattern in orchestrator function.

The second one is used to implement the **parallel execution** of the crawling workers, where a single worker is implemented as a separate activity function and the orchestrator function is responsible for invoking it multiple times in parallel, as shown in code snippet **4.2**.

```
@app.orchestration_trigger(context_name = "context")
def orchestrator_function(context: df.DurableOrchestrationContext):
        # retrieve URLs from the queue
        # filtering URLs to be crawled

        crawl_tasks = [context.call_activity("url_processor_function", msg)
                       for msg in url_to_crawl]
        crawl_results = yield context.task_all(crawl_tasks)

        # post-processing results
```

Code 4.2: Application of fan-out/fan-in pattern in orchestrator function.

Finally, the third one is used to implement the HTTP APIs that allow to interact with the crawling system, such as **starting a new crawl**, as shown in code snippet **4.3**.

```
@app.queue_trigger(arg_name = "azqueue",
                   queue_name = "queue-init",
                   connection = "AzureWebJobsStorage", )
@app.durable_client_input(client_name = "client")
async def crawling_starter(azqueue: func.QueueMessage, client) -> None:
    # retrieve parameters from the init queue
    # setup the environment for the crawling process

    logging.info(f"Starting orchestration with {max_workers} workers.")
    instance_id = await client.start_new("orchestrator_function",
                                         None, {"max_workers": max_workers,
                                                "max_depth": max_depth
                                         })
    logging.info(f"Launched orchestration with ID = '{instance_id}'.")
```

Code 4.3: Application of Async HTTP APIs pattern in orchestrator function.

4.5.2 Azure Storage constraints

The second problem related to the implementation of the crawling system refers to the storage resources needed to manage all the data generated during the crawling process.

Firstly, it was needed to create a **storage container** to store the crawled data separated by the marketplace and the depth of the crawling process, as shown in code snippet **4.4**. This allows to keep the data organized and easily accessible for further analysis or processing.

```
def upload_html(domain, page_name, content, container_name: str):
    logging.info(f"[UPLOAD HTML]: Connecting to storage container...")
    # connection to the storage container

    logging.info(f"[UPLOAD HTML]: Uploading HTML content to {container_name}...")
    blob_path = f"{domain}/{page_name}"
    blob_client = container_client.get_blob_client(blob_path)
    blob_client.upload_blob(content, overwrite = True)


upload_html(f"{MARKETPLACE_NAME}",
            f"level-{depth}/{hashlib.sha256(url.encode('utf-8')).hexdigest()}.html",
            response.text, "crawling-results")
```

Code 4.4: Saving crawled data to storage container.

Secondly, it was necessary to implement a queue to manage the URL frontier in a BFS-like manner. Given that storage costs for queues are relatively low compared to other services, and considering the **limitations in coordinating fine-grained access** by multiple concurrent workers, the queue is structured as a set of **BFS-level queues**,

where each queue corresponds to a specific **depth level** in the crawling process. This design enables efficient management of the URL frontier while remaining compatible with the serverless architecture and durable functions, as illustrated in code snippet **4.5**.

```
@app.route(route = "crawling_setup")
def crawling_setup(req: func.HttpRequest) -> func.HttpResponse:
    # retrieve parameters from the request
    # create the storage container for the crawling results

    queue_service = QueueServiceClient.from_connection_string(CONNECTION_STRING)

    logging.info(f"Creating {max_depth + 1} queues for BFS tree traversal...")
    for i in range(max_depth + 1):
        queue_name = f"{QUEUE_PREFIX}{i}"
        try:
            queue_service.create_queue(queue_name)
            logging.info(f"Created queue for level: {queue_name}")
        except:
            logging.info(f"Queue {QUEUE_PREFIX}{i} already exists.")

    # creates the utility table for storing environment variables

    return func.HttpResponse("Setup completed successfully!", status_code = 200)
```

Code 4.5: Creating BFS-level queues for URL frontier management.

Finally, the creation of a **utility table** was necessary to store the environment variables chosen by the user, since the stateless nature of the Azure Functions does not allow to store such information in a persistent way. The code snippet **4.6** shows how the utility table is created and used to store the environment variables.

```python
@app.route(route = "crawling_setup")
def crawling_setup(req: func.HttpRequest) -> func.HttpResponse:
    # retrieve parameters from the request
    # create the storage container for the crawling results
    # creates the BFS-level queues for URL frontier management

    table_service = TableServiceClient.from_connection_string(CONNECTION_STRING)
    table_client = table_service.get_table_client(TABLE_NAME)

    try:
        table_client.create_table()
        logging.info(f"Created table.")
    except:
        logging.info(f"Table {TABLE_NAME} already exists.")

    entity = {
        "PartitionKey": "Config",
        "RowKey": "GlobalSettings",
        "max_workers": max_workers,
        "max_depth": max_depth,
        "max_links": max_links,
        "max_exec_time": max_execution_time
    }
    table_client.upsert_entity(entity, mode = UpdateMode.REPLACE)
    logging.info("Configuration saved in Table Storage.")

    return func.HttpResponse("Setup completed successfully!", status_code = 200)
```

Code 4.6: Creating utility table for storing environment variables.

4.6 Anonymity and Security Measures

All HTTP requests made by the crawling workers are routed through a **containerized Tor proxy** deployed as an **Azure Container App**. The container is configured to expose a **SOCKS5 proxy**, and the crawling functions are designed to use this proxy endpoint. The container is built starting from a Docker image that includes the Tor service, which is configured to run in a **headless mode** and to listen on a specific port (**9050**) for incoming connections, as shown in code snippet **4.7**.

```
FROM debian:latest

RUN apt-get update && \
    apt-get install -y tor && \
    rm -rf /var/lib/apt/lists/*

COPY torrc /etc/tor/torrc
EXPOSE 9050

CMD ["tor", "-f", "/etc/tor/torrc"]
```

Code 4.7: Docker image for Tor proxy server.

In addition, other measures were implemented to enhance the security and anonymity of the crawling system:

- **Circuit Rotation**: the crawling workers are configured to periodically rotate the Tor circuit to enhance anonimity and reduce the risk of correlation attacks. This is achieved by sending a `SIGNAL NEWNYM` command to the Tor control port which triggers a new circuit creation, as shown in code snippet **4.8**.

```
1        def change_tor_ip():
2                with socket.create_connection(("tor-server-ip", 9051)) as s:
3                    s.sendall(b'AUTHENTICATE ""\r\nSIGNAL NEWNYM\r\nQUIT\r\n')
4                    s.close()
```

Code 4.8: Tor circuit rotation function.

- **Network Isolation**: the critical components of the architecture are deployed within a Virtual Network (**Azure VNet**) to ensure that all outbound traffic flows through the proxy. This prevents any direct access to the resources from the public internet and ensures that all the requests are sealed in a secure environment, as shown in figure **4.5**.

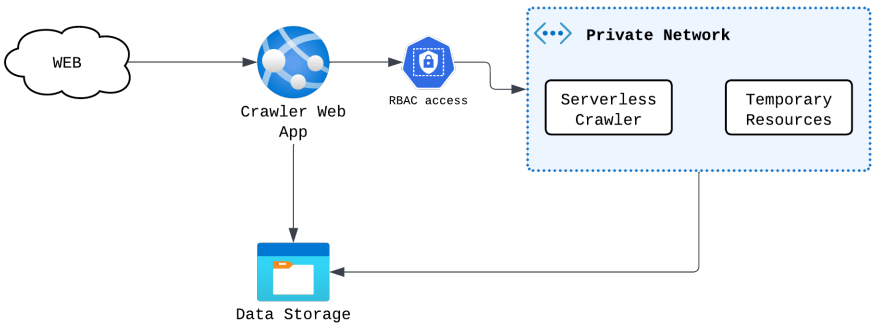

Figure 4.5: Virtual Network diagram.

- **Cookie Management**: a dedicated database is used to store cookies and session data, allowing the crawler to maintain state across multiple requests. This is particularly useful for crawling `.onion` services that require authentication or session management.

In addition to these measures, all the services and components are configured to be accessed only through RBAC (**Role-Based Access Control**) policies, ensuring that only authorized users and services can interact with the crawling system. This helps to prevent unauthorized access and potential data leaks.

4.7 Concurrency and Scalability Management

The serverless architecture used in this system is specifically **designed for scalability and resilience**, leveraging cloud-native services to efficiently handle the high volume of URLs and deep-link traversal typical of dark web crawling. It **automatically adapts** to fluctuating workloads without requiring manual resource provisioning, with changes only needed at the pricing plan level when demand increases significantly.

Additionally, the system provides **configurable parameters**, allowing users to tailor the crawling process according to specific requirements. These parameters include the **maximum number of concurrent workers**, the **maximum crawl depth**, and the total number of URLs to be processed. Such flexibility ensures that the crawler can be **fine-tuned** for different operational contexts—whether broad reconnaissance or targeted data extraction.

A containerized Tor proxy is integrated into the architecture to facilitate anonymous access to onion services. This proxy is also **designed to scale horizontally**, supporting up to three instances by default. Horizontal scalability ensures that multiple crawling workers can route their traffic through the Tor network simultaneously without causing network bottlenecks or latency spikes.

In summary, the architecture is designed for dynamic and elastic scaling, both at the application and network layers. This design choice is particularly critical for dark web crawling, where content volume, structure, and accessibility can vary drastically over time.

CHAPTER 5

Empirical Evaluation of Serverless CRATOR

This chapter outlines the experimental setup used for all the experiments presented in this thesis. It begins with two primary sections: the hardware specifications and the software tools employed to implement the distributed serverless architecture. Next, we detail the configuration used during the crawling process, the web crawlers selected for comparison with the serverless architecture, and the initial dataset. Finally, we present the metrics used to evaluate the performance of both the architecture and the crawlers.

5.1 Hardware specifications

Since the main goal is to design and develop a distributed serverless architecture, it is important to specify what the hardware specifications are. The architecture is designed to run on the **Microsoft Azure** cloud platform, which provides different pricing plans for each service, which can affect both performance and costs.

5.1.1 Pricing Plans

As mentioned earlier in this chapter, the pricing plans available for each service are different in terms of resources and costs. For this reason it's important to specify what are the ones used for this architecture, since they can affect both the **performance** of the crawler but also the **costs** of running the whole architecture. The last reason is particularly important, in fact there will be a section in the next chapters dedicated to the **cost analysis** of the architecture, which will take into account the pricing plans used for each service. For this particular implementation, the following pricing plans were used:

1. **Azure Functions**: the **consumption plan** was used for the experiments. This is a pay-as-you-go model that charges based on the number of executions and the execution time of the functions. Notably, it includes the first *1 million* executions per month at no cost. Other plans considered were the **flex consumption plan**, which provides on-demand and always-active resources, thereby mitigating cold starts, and the **premium plan**, which offers enhanced performance and additional features at a higher cost. Table 5.1 summarizes the key differences among these plans.

Feature	Consumption Plan	Flex Consumption Plan	Premium Plan
Billing Model	Pay-per-execution	On-demand and Always Ready instances	Dedicated instances with pre-warmed capacity
Free Grant (per month)	1 million executions, 400,000 GB-s	250,000 executions, 100,000 GB-s (On-Demand only)	None
Execution Cost	€0.176 per million executions	€0.352 per million executions (On-Demand)	Included in plan
Execution Time Cost	€0.000015 per GB-s	€0.000023 per GB-s (On-Demand)	Included in plan
Always Ready Baseline Cost	N/A	€0.0000036 per GB-s	N/A
Always Ready Execution Cost	N/A	€0.0000141 per GB-s	N/A
Cold Start	Yes	Reduced (with Always Ready)	No
Maximum Execution Duration	5 minutes	60 minutes	Unlimited
Scaling	Automatic (event-driven)	Automatic with concurrency settings	Automatic with pre-warmed instances
Dedicated Instances	No	Optional (Always Ready)	Yes (required)

Table 5.1: Azure Functions pricing plan comparison (North Europe region)

2. **Azure Storage**: for the creation of the storage account, the **standard v2** performance tier was used, which is the most cost-effective option for general-purpose storage. The pricing is based on the amount of data stored and the number of operations performed on the data.

3. **Azure CosmosDB**: the **serverless** capacity mode plan was used. In this pay-per-request plan, the cost are related to the resources consumed by the database, which means that you are charged based on the number of **requests units** and the amount of data stored. Table **5.2** summarizes the key differences among the available plans, including the **provisioned throughput** and **autoscale provisioned** plans, which are designed for applications with more predictable workloads.

Feature	Serverless	Provisioned Throughput	Autoscale Provisioned
Billing Model	Pay-per-request (RU/s consumed)	Pre-allocated RU/s	Auto-adjusting RU/s
Request Unit Cost	€0.249 per million RU/s	€0.0071 per 100 RU/s-hour	€0.008 per 100 RU/s-hour
Storage Cost	N/A	€0.25 per GB/month	€0.25 per GB/month
Scaling	Up to 5,000 RU/s per container	Manual (min 400 RU/s)	Auto (scales 10%–100% of max)
Cold Start	No cold start	Always available	Always available
Best For	Intermittent, small workloads	Steady workloads with known traffic	Variable workloads with peak demand

Table 5.2: Azure CosmosDB pricing plan comparison (North Europe region)

4. **Azure Container Registry**: this service is necessary to store the container images used by the Azure Container Apps service. The **basic tier** was used, while the other tiers, such as **standard** and **premium**, are designed for larger-scale applications with higher performance requirements. Table **5.3** summarizes the key differences among the available plans.

Feature	Basic	Standard	Premium
Price per Day	€0.147	€0.586	€1.465
Included Storage	10 GB	100 GB	500 GB
Additional Storage Cost	€0.00293/GB-day	€0.00293/GB-day	€0.00293/GB-day
Container Build	€0.00009/CPU-second of task running	€0.00009/CPU-second of task running	€0.00009/CPU-second of task running
Read Operations per Minute	1,000	3,000	10,000
Write Operations per Minute	100	500	2,000
Download Bandwidth (Mbps)	30	60	100
Upload Bandwidth (Mbps)	10	20	50
Webhooks	2	10	500

Table 5.3: Azure Container Registry pricing plan comparison (North Europe region)

5. **Azure Container Apps**: the **consumption plan** was used, which is a pay-as-you-go model that charges based on the number of container instances and the resources consumed by the containers. Compared to the **dedicated**

plan, which is designed for applications with more predictable workloads, the consumption plan is more cost-effective for applications with variable workloads, such as a web crawler. The consumption plan allows to scale the number of container instances automatically based on the workload, which is ideal for applications that require high scalability and flexibility. Table **5.4** summarizes the key differences among the available plans.

Feature	Consumption Plan	Dedicated Plan
Billing Model	Pay-per-use (per vCPU-second, GiB-second, and request)	Fixed pricing based on provisioned resources
Free Monthly Grants	180,000 vCPU-seconds, 360,000 GiB-seconds, 2 million requests	Not applicable
vCPU Pricing	Active: €0.0000211/sec; Idle: €0.0000027/sec	€0.0502/hour
Memory Pricing	€0.0000027 per GiB-second	€0.0044/hour
Request Pricing	€0.352 per million	Included in fixed pricing
Scaling	Automatic scaling to zero	Manual scaling; always-on instances
Best For	Event-driven, intermittent workloads	High-performance, always-on applications

Table 5.4: Azure Container Apps pricing plan comparison (North Europe region)

6. **Azure KeyVault**: the pricing plan used for this service is the **standard tier**, as the additional features offered by the **premium tier**, such as Hardware Security Modules (HSMs) and advanced access policies, are not required. The standard tier provides the same core capabilities as the premium tier for storing and managing secrets, keys, and certificates.

7. **Azure Web App**: the pricing plan used for this service is the **Basic B1** with Linux as operating system, which provides a single instance with 1 vCPUs and 1.75 GB of RAM, which is sufficient for running the web application used in this architecture. The other plans, such as **Premium v3** and **F1 (Free)**, are designed for applications with higher performance requirements or for testing purposes. Table **5.5** summarizes the key differences among the available plans.

Feature	F1 (Free)	B1-B3 (Basic)	Premium v3
vCPU	Shared	1-4 vCPU	1-32 vCPU
RAM	1 GB	1.75-7 GB	4-256 GB
Storage	1 GB	10 GB	250 GB
Max Instances	1	Up to 3	Up to 30
Price (€/hour)	Free	€0.016-€0.062	€0.074-€2.835

Table 5.5: Azure App Service Linux pricing plan comparison (North Europe region)

Other services, such as **Azure VNet** and **Azure SQL Database** are not included in this section as they don't have specific pricing plans. In fact, the costs of the virtual network are based on the amount of data transferred and the number of virtual network gateways used, while the costs of the Azure SQL Database are based on the amount of data stored and the number of requests made to the database.

5.1.2 On-Premise Hardware

To provide a **baseline** for comparison with the proposed serverless crawler architecture, the results concerning a traditional on-premise implementation of web crawlers were taken directly from the reference work by De Pascale *et al.*[5]. This data serves as a baseline to directly compare performance, scalability, and resource usage metrics against the cloud-native solution.

The on-premise environment described in the reference paper was used to run traditional crawlers, which did not leverage any serverless capabilities, and were executed sequentially or with parallelism on a single host machine.The hardware specifications of the machine used for those experiments are the following:

- **CPU**: Intel(R) Core(TM) i7-9700 CPU @ 3.00GHz
- **RAM**: 16GB 2666MT/s DDR4 RAM
- **Operating System**: Ubuntu 22.04

These specifications are reported to contextualize the performance measurements and resource utilization of the on-premise crawlers as a comparative reference point.

5.2 Programming Languages and Software Libraries

For the implementation of the distributed serverless architecture, several software tools and technologies were employed. The entire system was primarily developed using **Python 3.11.11**, chosen for its simplicity, readability, and the vast ecosystem of libraries and frameworks that support cloud-native and data processing applications. Moreover, Python is natively supported by the Azure Functions runtime, which significantly simplifies deployment and integration within the Azure cloud infrastructure.
A number of libraries were used to implement different components of the system. The most relevant ones include:

- **azure-core**, **azure-functions-***, **azure-storage-***: these libraries provide interfaces for interacting with various Azure services such as Azure Functions, Blob Storage, Queue Storage, and Table Storage.
- **pymongo**: used to interact with the NoSQL database hosted on Azure Cosmos DB with the MongoDB API.
- **requests**: a widely-used library for sending HTTP requests. In this architecture, it is used to perform GET requests to retrieve web page content from `.onion` sites over the Tor network.
- **beautifulsoup4**: employed for parsing and extracting hyperlinks from HTML documents. This library is essential for link discovery during the crawling process.
- **Flask**: used to develop a lightweight web interface for the crawler. The interface allows users to initiate crawl tasks, monitor progress, and manage configuration settings.

In addition to the programming language and libraries, proper **version control** and **continuous integration** were essential for managing the codebase and automating deployments. To this end, **Git** was used for source control, while **GitHub Actions** was employed to configure a *Continuous Integration/Continuous Deployment (CI/CD)*

pipeline. This pipeline facilitates automated testing, builds, and deployments, ensuring that the system remains consistent, maintainable, and easily extensible throughout the development lifecycle.

5.3 Crawling Testing and Configuration

This section is focused on the specifics of the test configurations and the types of tests performed on the new crawling architecture. The results obtained from these experiments will then be used for a direct comparison with the performance of the **on-premise baseline** reference.

5.3.1 Test Combinations

The experimental configurations were defined by crossing two fundamental parameters that directly influence the workload, the scanning depth, and the concurrency of the crawling system:

- **Crawling Depth (D):** it specifies the maximum depth of link discovery beyond which the crawler will not proceed in exploring the site. The depth levels considered are $D \in \{1, 2, 3\}$.
- **Worker Count (W):** it defines the number of concurrent workers that will be activated for the crawling execution. This parameter is crucial for testing the horizontal scalability of the system. The values used for testing are $W \in \{5, 10\}$.

The intersection of these two parameters (D, W) generated a total of six (**6**) unique test combinations, allowing us to analyze the joint impact of target complexity (depth) and system scalability (concurrency). The combinations are summarized as follows:

1. **Depth 1 - 5 Workers** $(D = 1, W = 5)$
2. **Depth 2 - 5 Workers** $(D = 2, W = 5)$
3. **Depth 3 - 5 Workers** $(D = 3, W = 5)$
4. **Depth 1 - 10 Workers** $(D = 1, W = 10)$

5. **Depth 2 - 10 Workers** ($D = 2, W = 10$)

6. **Depth 3 - 10 Workers** ($D = 3, W = 10$)

5.3.2 Testing Strategies

For this experimentation, two distinct types of test were conducted with the aim of evaluating the **effectiveness** and **reliability** of the proposed solution in both the **short** and **long term**. For each of the six configuration combinations identified (Depth, Workers), a total of **10 test runs** were performed. The final results reported for each measured metric will represent the average of these 10 runs.

Bulk Test (Immediate Reliability Test)

The Bulk Test was designed to measure the reliability and variance of the crawler under **near-real-time** operating conditions. Therefore, the goal is to measure performance over a limited period of time in order to analyze **operational consistency** and the impact of short-term load. All 10 test runs for a given combination were performed **continuously** and **sequentially**, minimizing the time between the end of one run and the start of the next, thereby simulating an intensive and concentrated load. The comparison of results was conducted by calculating the *mean* of the measured metrics (e.g., execution time, throughput, ecc...) and the corresponding *standard deviation* (σ). A low standard deviation ($\sigma \approx 0$) serves as a primary indicator of greater stability and reliability of the system under immediate and concentrated operational conditions.

Temporal Test (Long-Term Stability Test)

The Temporal Test was conducted with the opposite goal: measuring crawler behavior over an extended time window to **detect fluctuations** due to external factors (e.g., cloud platform load, network congestion, peak times).
The 10 test runs for each specified combination were performed with a deliberate time separation, i.e., once a day for ten consecutive working days (monday through friday). This execution pattern allowed us to **evaluate fluctuations** in execution times

and results in a realistic operating environment. The goal was to isolate and quantify the impact of the intrinsic variability of network and cloud conditions.

5.3.3 Metrics for the evaluation

The metrics used to compare the new architecture and the on-premise CRATOR crawler were defined following the methodological framework proposed in the paper by De Pascale *et al.* [5]. In addition, more details regarding the **operating costs** of the cloud platform and new **temporal metrics** were integrated.

For these reasons, the key metrics taken in consideration for the evaluation are:

- **Coverage**, which, in the context of crawling, expresses a *crawler's ability to successfully extract data from a defined set of input sources*. In this experiment, coverage will be measured by tracking the average number of **pages downloaded** for each depth level and the **ratio** of the number of pages successfully extracted to the number of pages discovered.

- **Performance**, evaluated in terms of time efficiency and productivity. In particular, **execution time** and **download rate**, expressed as pages downloaded per minute (**PPM**), were measured.

- **Robustness**, defined as *the capability of a crawler to continue operating even in the presence of failures or errors*. In this case, robustness was evaluated using the **error rate**, e.g., the number of pages not downloaded compared to the total number of pages found.

5.3.4 Case Scenario

In order to evaluate serverless architecture, *Cocoriko-Market* [50], a well-known **European marketplace**, was selected as the reference environment. This marketplace is characterized by anonymization and privacy mechanisms that allow it to be used for the sale and purchase of illegal goods and services.

This environment stands out in particular for its access management: it does **not require mandatory login**, making access to the site's resources completely free and available to anyone. Due to this mechanism, which guarantees open access, it has

not been necessary to **use cookie authentication** for simple navigation and page downloads.

5.4 Data Analysis and RQ Mapping

To clearly establish how the experimental setup addresses the research objectives defined in Chapter 1, this section explicitly maps the collected metrics to each Research Question (RQ).

- **Mapping for RQ_1 (Feasibility):** The feasibility is assessed by analyzing the *Architecture Design* (presented in Chapter 4) in conjunction with the *Coverage* and *Error Rate* metrics. These show if the serverless environment can effectively discover and process onion services despite the instability of the Tor network.
- **Mapping for RQ_2 (Key Differences):** The comparison between the On-Premise and Serverless models is driven by *Performance metrics* (Execution Time, Pages per Minute, Download Time) and *Economic metrics* (Cost Analysis and Break-even point). These metrics highlight the trade-offs between raw local speed and cloud-native elasticity.
- **Mapping for RQ_3 (Accessibility):** The answer to this RQ lies in the implementation of the *Web Interface* and *Asynchronous APIs*. The ability for a user to trigger and monitor a crawl without local dependencies is the primary metric for evaluating the tool's accessibility.

CHAPTER 6

Results and Discussion

This chapter illustrates the data collected during the experiments, highlighting the performance differences between the cloud architecture and the on-premise crawler. To provide a clear evaluation of the proposed system, the presentation of the data is organized by Research Question. For each topic, the results obtained from both the Bulk Tests (immediate reliability) and Temporal Tests (long-term stability) are clearly distinct and analyzed, concluding with a detailed analysis of the economic results to properly assess the trade-offs of the cloud solution.

6.1 RQ1: Feasibility and Reliability of a Serverless Crawler

This section evaluates the system's ability to successfully discover and download dark-web pages, maintaining high coverage and low error rates despite the Tor network's instability.

6.1.1 Coverage Comparison

Bulk Test Results

The quantitative results relating to coverage are presented in table **6.1**.

		Depth 1			Depth 2			Depth 3		
		Total Objects	**Crawled True**	**Rate**	**Total Objects**	**Crawled True**	**Rate**	**Total Objects**	**Crawled True**	**Rate**
Crator	mean	115.9	115.9	1	622.2	622.1	1	2266.5	2266.5	1
	std	0.316	0.316		12.2	12.2		79.271	79.271	
Serverless Crator 5W	mean	114	113.7	0.9974	583.5	581.7	0.9969	2086.9	2078.1	0.9958
	std	0	0.6403		24.6830	24.6741		170.3229	169.8054	
Serverless Crator 10W	mean	114	113.7	0.9974	593.8	592.8	0.9983	2001.9	1967.2	0.9827
	std	0	0.9		18.7286	19.3845		131.0324	128.3813	

Table 6.1: Bulk Test coverage and relative coverage results of CRATOR and Serverless Crator with 5 and 10 workers.

As a visual reinforcement, figures **6.1** and **6.2** show the data presented in table **6.1**.

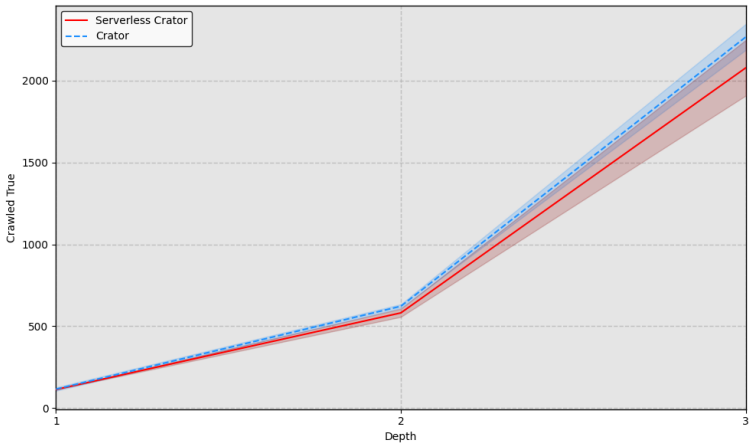

Figure 6.1: Bulk Test coverage comparison between CRATOR and Serverless Crator with 5 workers.

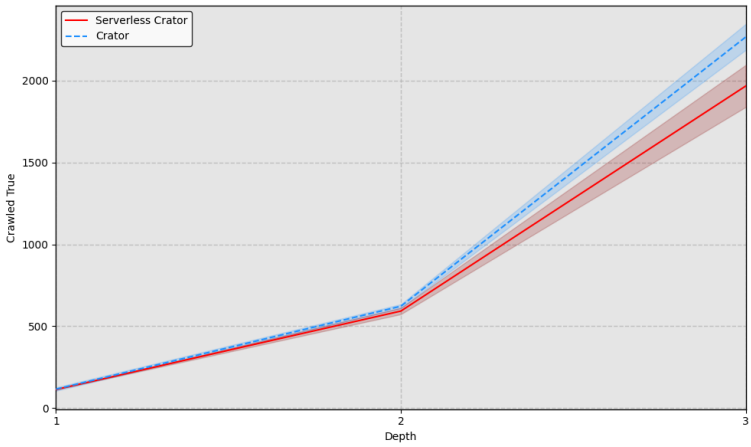

Figure 6.2: Bulk Test coverage comparison between CRATOR and Serverless Crator with 10 workers.

In the first two levels, **the difference** in the number of pages downloaded correctly between the CRATOR system and Serverless Crator **is marginal**. The differences begin to appear at the third level of depth:

- CRATOR shows **less variance** and a **higher overall** number of pages downloaded.
- Serverless Crator shows **greater variance** in the results obtained. It was observed that increasing the number of workers to 10, while leading to a slightly lower average number of pages downloaded, also ensured greater stability and consistency in execution compared to the 5-worker configuration.

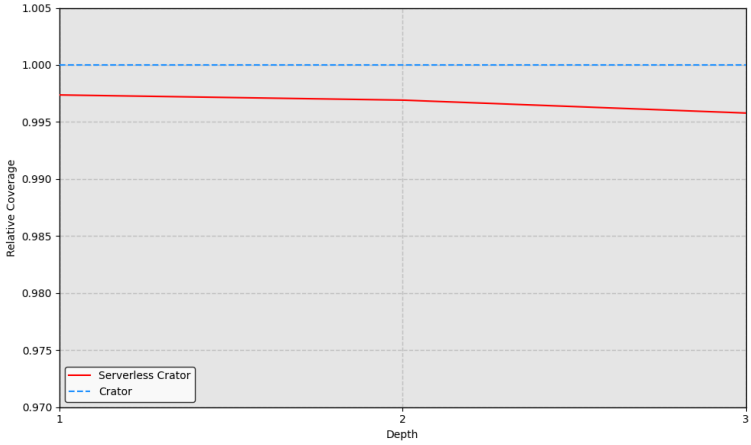

Figure 6.3: Bulk Test Relative Coverage comparison between CRATOR and Serverless Crator with 5 workers.

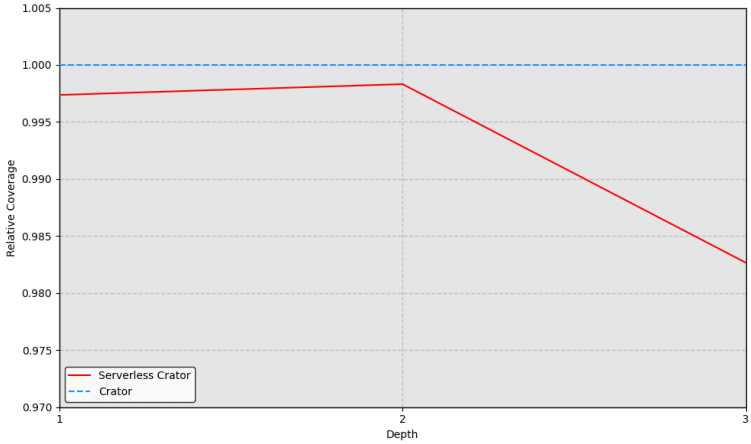

Figure 6.4: Bulk Test relative coverage comparison between CRATOR and Serverless Crator with 10 workers.

In terms of relative coverage (figures **6.3** and **6.4**), CRATOR demonstrates perfect performance (1.0 ratio). Serverless Crator also exhibits an **excellent rate**, consistently hovering around **98%** at all levels analyzed.

Temporal Test Results

The quantitative results relating to coverage are presented in table **6.2**.

		Depth 1			Depth 2			Depth 3		
		Total Objects	**Crawled True**	**Rate**	**Total Objects**	**Crawled True**	**Rate**	**Total Objects**	**Crawled True**	**Rate**
Crator	mean	115.9	115.9	1	622.2	622.2	1	2266.5	2266.5	1
	std	0.316	0.316		12.2	12.2		79.271	79.271	
Serverless Crator 5W	mean	114.0	113.8	0.9982	587.0	585.6	0.9976	2097.9	2090.4	0.9964
	std	0	0.6		19.6418	20.4020		171.8688	171.8949	
Serverless Crator 10W	mean	114.0	114.0	1	590.0	588.2	0.9969	2091.4	2084.0	0.9965
	std	0	0		20.9476	21.7200		167.3399	168.7193	

Table 6.2: Temporal Test coverage and relative coverage of CRATOR and Serverless Crator with 5 and 10 workers.

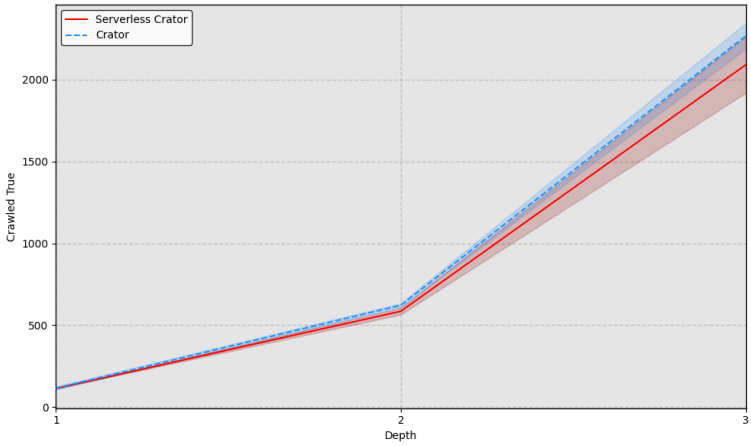

Figure 6.5: Temporal Test coverage comparison between CRATOR and Serverless Crator with 5 workers.

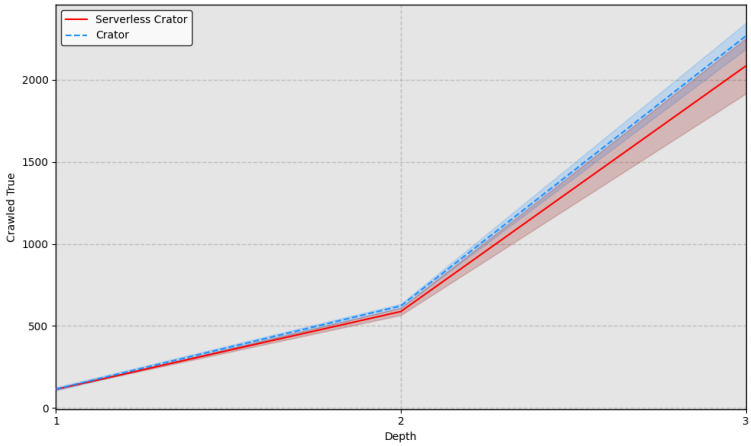

Figure 6.6: Temporal Test coverage comparison between CRATOR and Serverless Crator with 10 workers.

The differences in crawler performance are shown in figures **6.5** and **6.6**. In contrast to the Bulk Test, the **discrepancy** between crawlers in terms of pages discovered is **less pronounced**. Running the process over a longer period showed **significant improvements** in Serverless Crator's performance.

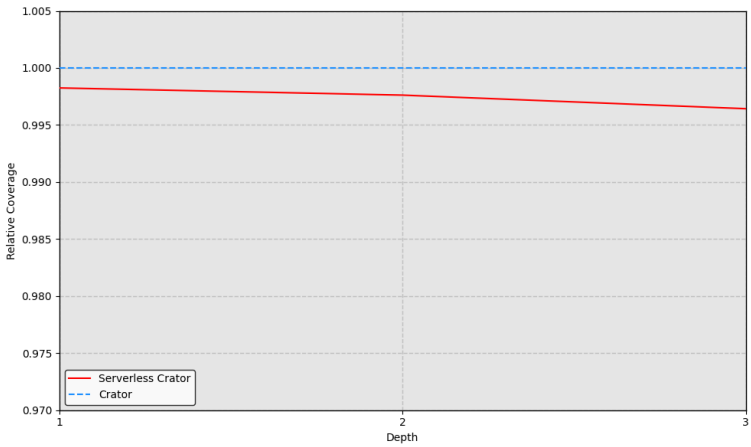

Figure 6.7: Temporal Test relative coverage comparison between CRATOR and Serverless Crator with 5 workers.

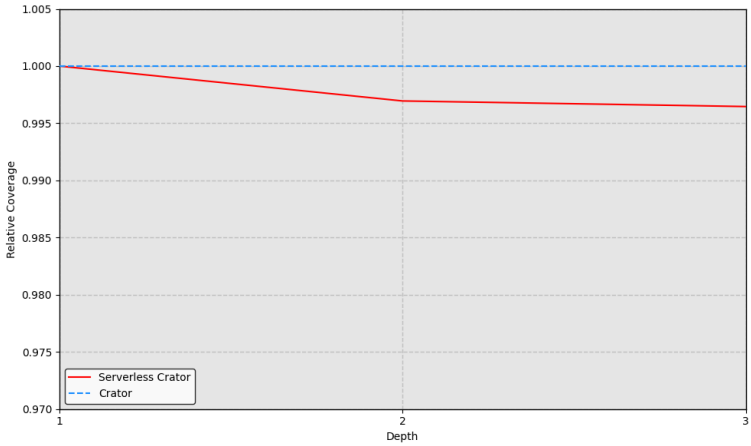

Figure 6.8: Temporal Test relative coverage comparison between CRATOR and Serverless Crator with 10 workers.

The relative coverage data (figures **6.7** and **6.8**) confirms that the percentage of links downloaded correctly is **always close to 99%** over time. It is important to note that each execution was conducted at approximately the same times of day. This opens up the possibility for **further analysis** of the impact of network and platform load during specific times of day on the discovery metrics of the two serverless crawlers.

In summary, the results show that while coverage performance is **comparable at low depths**, as scan complexity increases, the CRATOR system demonstrates **greater stability and completeness**. Serverless Crator, while maintaining overall excellent relative coverage, exhibits **greater variability in performance** as depth increases.

6.1.2 Robustness and Error Rate

Bulk Test Results

The quantitative results relating to robustness are presented in table **6.3**.

		Depth 1			Depth 2			Depth 3		
		Total Objects	**Crawled False**	**Rate**	**Total Objects**	**Crawled False**	**Rate**	**Total Objects**	**Crawled False**	**Rate**
Crator	mean	115.9	0	0	622.2	0	0	2266.5	0	0
	std	0.316	0		12.2	0		79.271	0	
Serverless Crator 5W	mean	114	0.3	0.0026	583.5	1.8	0.0031	2086.9	8.8	0.0042
	std	0	0.6403		24.6830	1.7205		170.3229	10.1074	
Serverless Crator 10W	mean	114	0.3	0.0026	593.8	1.0	0.0017	2001.9	34.7	0.0173
	std	0	0.9000		18.7286	1.7889		131.0324	81.3671	

Table 6.3: Bulk Test robustness results of CRATOR and Serverless Crator with 5 and 10 workers.

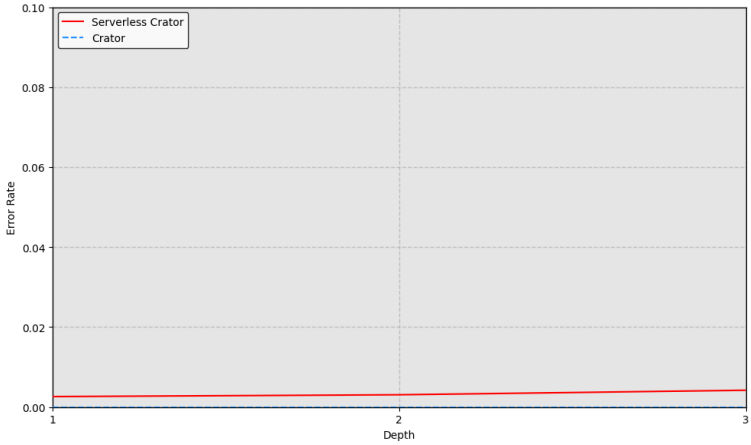

Figure 6.9: Bulk Test error-rate comparison between CRATOR and Serverless Crator with 5 workers.

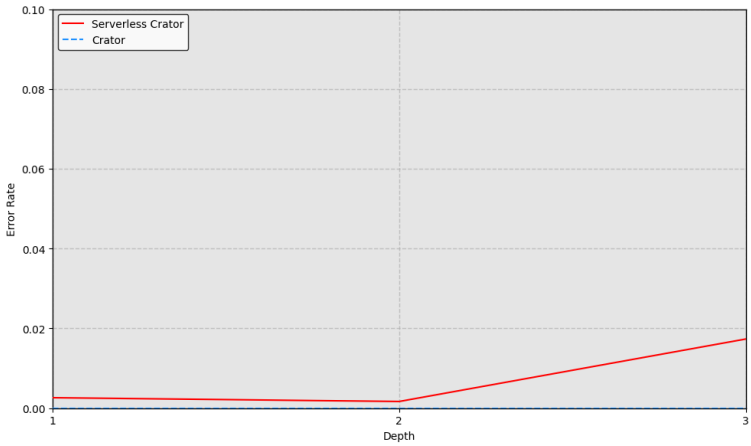

Figure 6.10: Bulk Test error-rate comparison between CRATOR and Serverless Crator with 10 workers.

In terms of reliability, while CRATOR crawler executions remain consistently

perfect with an error rate of 0%, Serverless Crator also **demonstrates excellent robustness**. Both configurations tested (5 and 10 workers) have an **error rate of around 0%**.

There was a slight increase in the error rate at depth level 3 for the 10-worker configuration. This anomaly is attributed to an unexpected interruption of a test, caused by a cloud platform error, which terminated the execution before completion and consequently left a large number of links unanalyzed in the execution.

Temporal Test Results

		Depth 1			Depth 2			Depth 3		
		Total Objects	**Crawled False**	**Rate**	**Total Objects**	**Crawled False**	**Rate**	**Total Objects**	**Crawled False**	**Rate**
Crator	mean	115.9	0	0	622.2	0	0	2266.5	0	0
	std	0.316	0		12.2	0		79.271	0	
Serverless Crator 5W	mean	114.0	0.2	0.0018	587.0	1.4	0.0024	2097.9	7.5	0.0036
	std	0	0.6		19.6418	1.8		171.8688	4.4777	
Serverless Crator 10W	mean	114.0	0	0	590.0	1.8	0.0031	2091.4	7.4	0.0035
	std	0	0		20.9476	2.6		167.3399	41.28	

Table 6.4: Temporal Test robustness results of CRATOR and Serverless Crator with 5 and 10 workers.

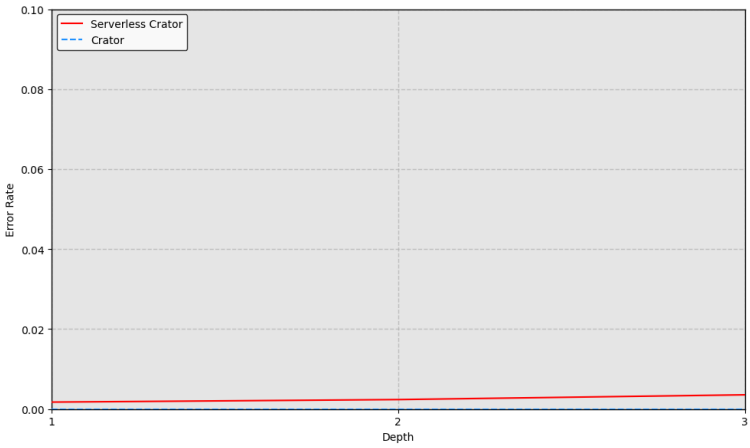

Figure 6.11: Temporal Test error-rate comparison between CRATOR and Serverless Crator with 5 workers.

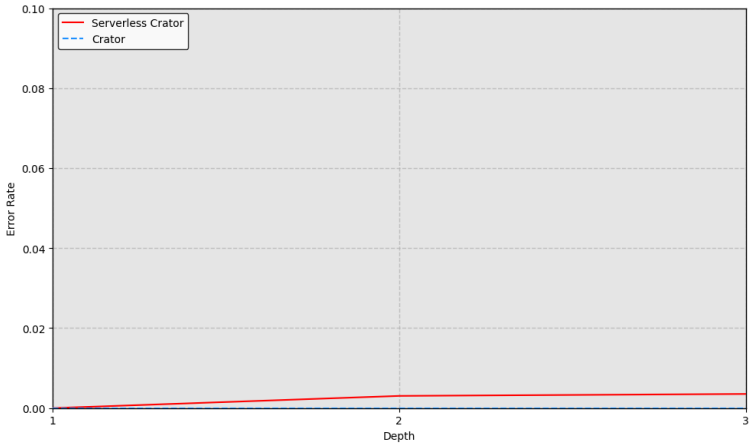

Figure 6.12: Temporal Test error-rate comparison between CRATOR and Serverless Crator with 10 workers.

The same type of improvement seen in coverage is also noticeable in robustness. Error rates have **fallen further** over time, bringing the error rate of Serverless Crator **significantly closer to the ideal 0**.

6.2 RQ2: On-Premise vs Serverless: Key Differences

This section analyzes the performance metrics related to Execution Time, Pages per Minute and Download Time.

6.2.1 Execution Time and Pages per Minute

Bulk Test Results

The quantitative results relating to performace are presented in table **6.5**.

		Depth 1		Depth 2		Depth 3	
		Execution Time	**Pages per Minute**	**Execution Time**	**Pages per Minute**	**Execution Time**	**Pages per Minute**
Crator	mean	173.3	28.975	939.2	37.276	3420.6	39.076
	std	1.418	0.079	18.103	0.929	119.295	0.280
Serverless Crator 5W	mean	124.2	56.3595	1705.4	20.4861	10540.1	11.8557
	std	19.8736	9.0309	65.1233	0.9774	1027.8225	0.3815
Serverless Crator 10W	mean	100.8	69.3519	1439.8	24.7118	7429.1	15.8930
	std	14.6479	11.7499	41.6144	0.7481	471.8575	0.4857

Table 6.5: Bulk Test performance results of CRATOR and Serverless Crator with 5 and 10 workers.

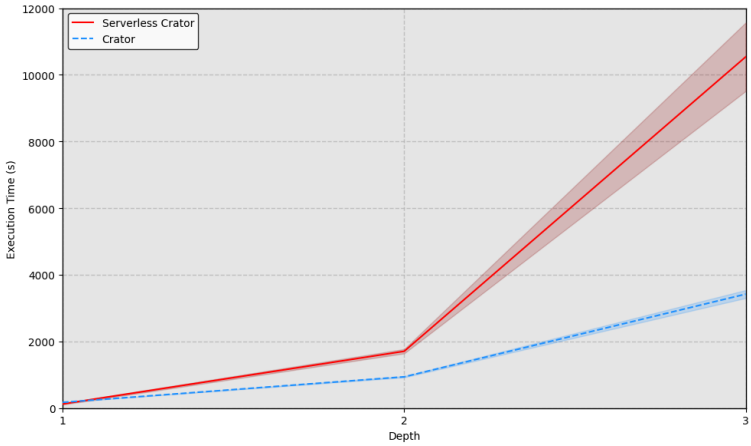

Figure 6.13: Bulk Test execution time comparison between CRATOR and Serverless Crator with 5 workers.

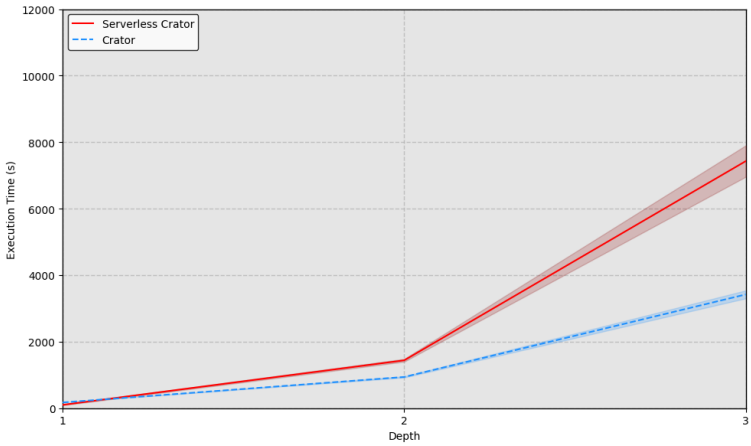

Figure 6.14: Bulk Test execution time comparison between CRATOR and Serverless Crator with 10 workers.

In terms of execution time, the results show that the CRATOR system is **indisputably faster than both Serverless Crator configurations** (with 5 and 10 workers). However, there is a clear **progressive improvement** in execution times as the **number of workers** in the serverless environment **increases**, suggesting much margin for further optimization. It is essential to consider that Serverless Crator operates in a cloud environment, where execution times are inevitably affected by service latency, message exchange waiting times, and connection dynamics, factors that do not affect the local execution of CRATOR.

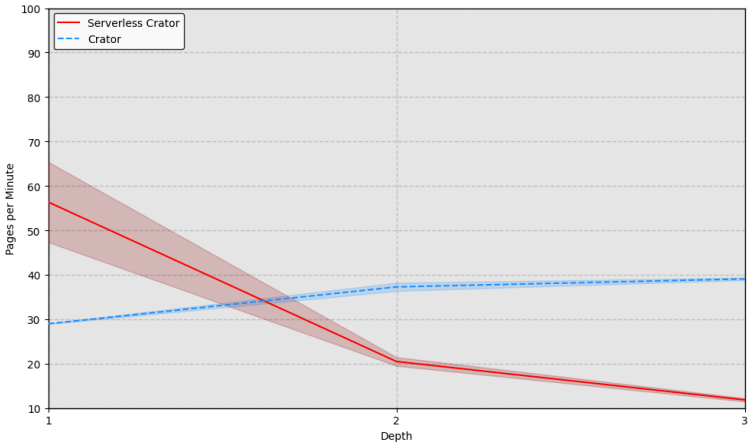

Figure 6.15: Bulk Test PPM comparison between CRATOR and Serverless Crator with 5 workers.

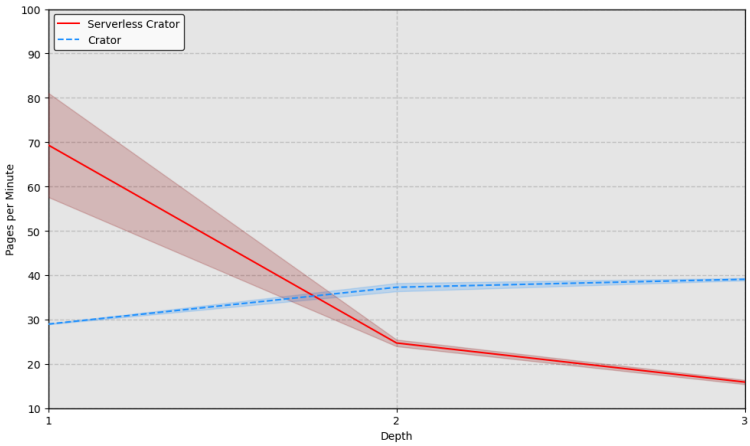

Figure 6.16: Bulk Test PPM comparison between CRATOR and Serverless Crator with 10 workers.

The differences in speed are also obvious in the Pages Per Minute (**PPM**) metric. We see a clear drop in Serverless Crator's performance when going from depth 2 to 3. However, it is worth noting the anomaly recorded at level 1, where Serverless Crator, with both 5 and 10 workers, shows a significantly higher number of links downloaded. This result, in apparent contrast to the overall execution times, is closely related to the Time-to-Download metric of the links themselves, which will be explained in detail in the following sections.

Temporal Test Results

The quantitative results relating to performace are presented in table **6.6**.

		Depth 1		Depth 2		Depth 3	
		Execution Time	**Pages per Minute**	**Execution Time**	**Pages per Minute**	**Execution Time**	**Pages per Minute**
Crator	mean	173.3	28.975	939.2	37.276	3420.6	39.076
	std	1.418	0.079	18.103	0.929	119.295	0.280
Serverless Crator 5W	mean	128.4	54.0228	1690.2	20.8765	10448.8	12.1496
	std	16.4815	6.6746	130.8761	1.2694	1614.4450	1.0190
Serverless Crator 10W	mean	104.6	69.8634	1456.7	24.2481	7719.8	16.2149
	std	23.9132	20.4282	36.5569	1.1810	687.8239	0.4271

Table 6.6: Temporal Test performance results of CRATOR and Serverless Crator with 5 and 10 workers.

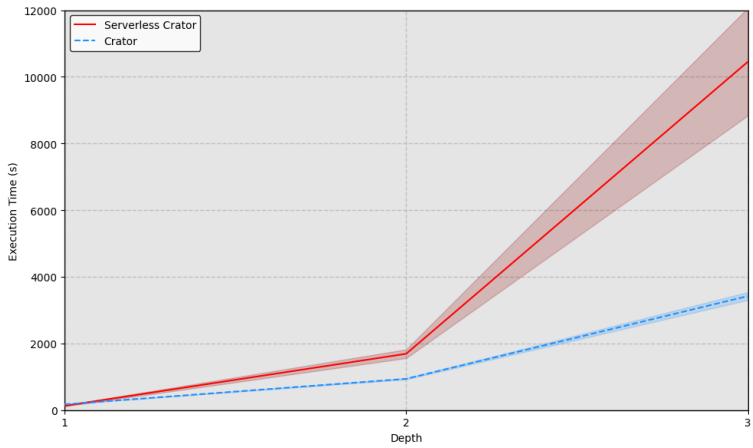

Figure 6.17: Temporal Test execution time comparison between CRATOR and Serverless Crator with 5 workers.

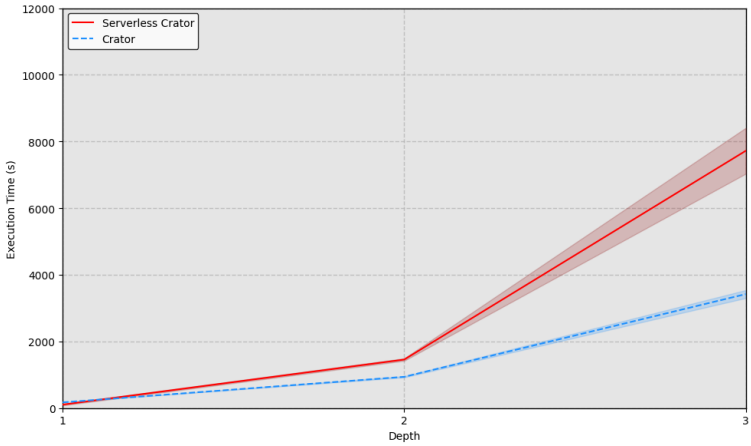

Figure 6.18: Temporal Test execution time comparison between CRATOR and Serverless Crator with 10 workers.

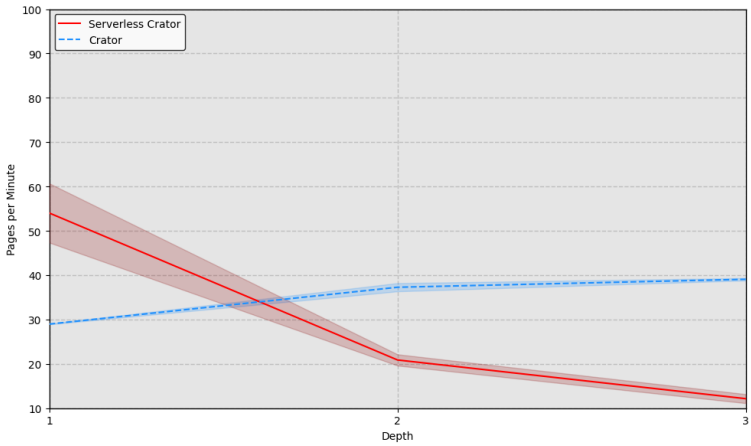

Figure 6.19: Temporal Test PPM comparison between CRATOR and Serverless Crator with 5 workers.

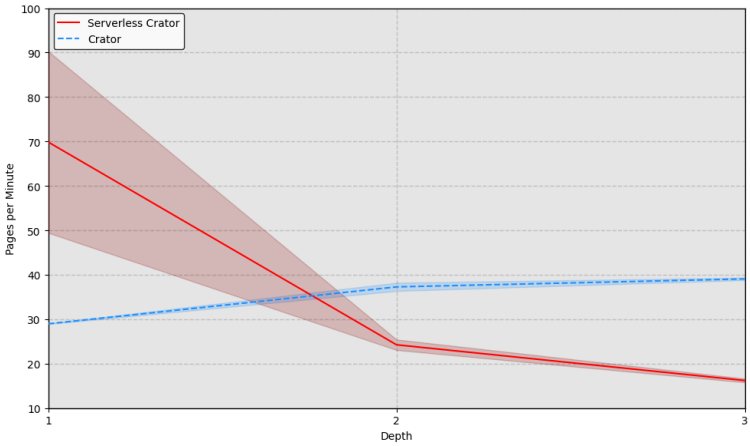

Figure 6.20: Temporal Test PPM comparison between CRATOR and Serverless Crator with 10 workers.

The execution times and PPM values over the Temporal Test are **consistent with** what we noticed in the **Bulk Test**, confirming that the on-premise execution preserves a higher throughput.

6.2.2 Download Time Analysis

Bulk Test Results

In order to get a deeper understanding of the critical issues and common behaviors of Serverless Crator, the Download Time metric has been introduced. This metric is specifically designed to *analyze the average download times recorded for links at each level of depth*. This analysis is crucial for identifying the levels where the longest download times occur and for tracing the specific causes of these slowdowns. Table **6.7** summarizes the average download times recorded during test runs.

			Depth 0		Depth 1		Depth 2		Depth 3	
	Global Mean	**Global Std**	**Mean**	**Std**	**Mean**	**Std**	**Mean**	**Std**	**Mean**	**Std**
1D - 5W	2.0637	0.1102	3.0657	5.3262	2.0601	0.1012				
1D - 10W	1.9011	0.1431	1.3220	0.5448	1.9109	0.1346				
2D - 5W	2.4738	0.2182	8.2782	13.6481	2.4589	0.2945	2.4748	0.2590		
2D - 10W	2.1534	0.1472	1.3695	0.3524	2.1429	0.1194	2.1613	0.1697		
3D - 5W	4.3205	0.4429	11.1917	12.9206	4.5314	0.6725	4.3969	0.5407	4.2972	0.4342
3D - 10W	2.3948	0.2826	10.7863	14.4219	2.4825	0.3212	2.5107	0.3069	2.3916	0.2245

Table 6.7: Bulk Test download time results for Serverless Crator.

The overall average download time per link remains **remarkably consistent**, settling at **around 2.55s**. However, it is essential to put the spotlight on the results recorded in the **Depth 0 column**. The average times at this level are **significantly higher** than the other values. This discrepancy is explained by the fact that the root URL, which initiates the crawling process, has to endure **longer connection** and **response times** with the Tor container, as it represents the first request of the session. This behavior is particularly evident in depth 3 runs, where the average values for **level 0 reach around 10s**, fully justifying the higher initial latency.

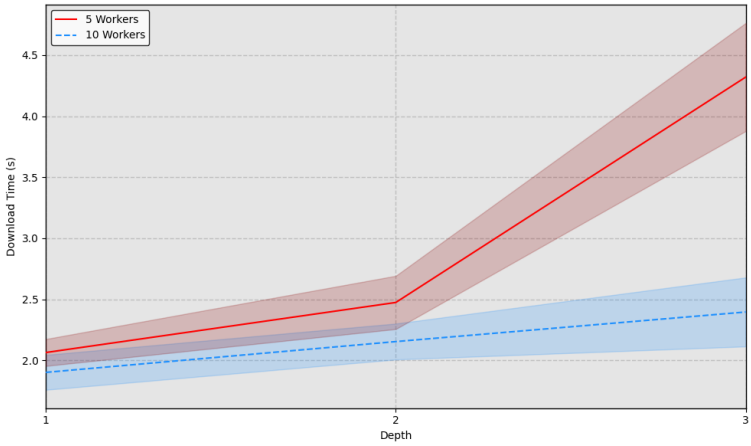

Figure 6.21: Bulk Test download time of Serverless Crator.

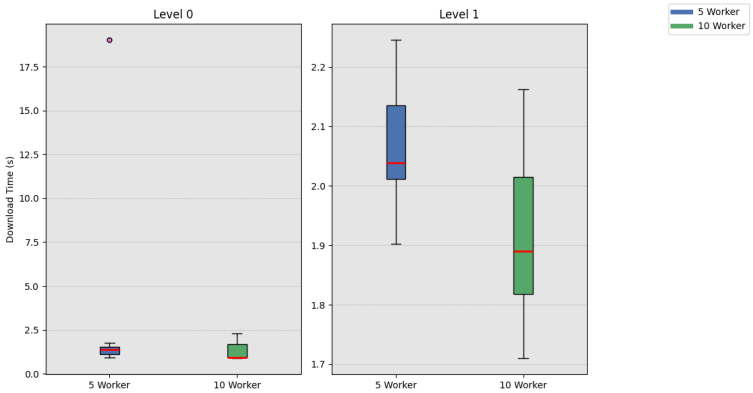

Figure 6.22: Bulk Test download time of Serverless Crator at level 1.

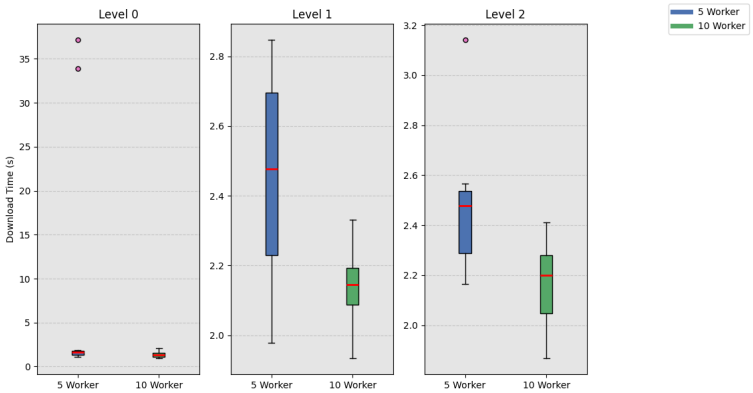

Figure 6.23: Bulk Test download time of Serverless Crator at level 2.

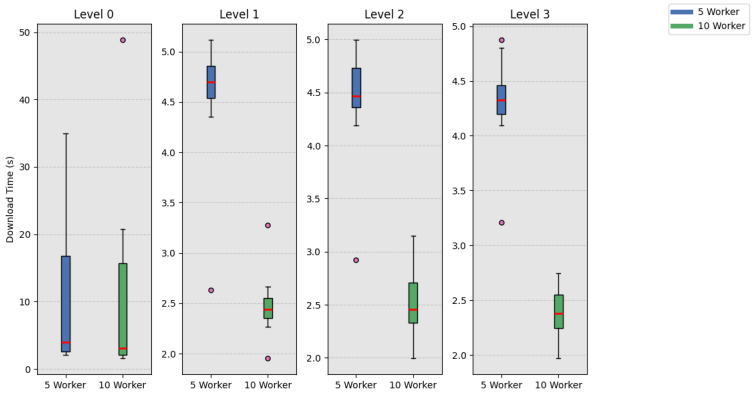

Figure 6.24: Bulk Test download time of Serverless Crator at level 3.

Figures **6.22**, **6.23**, and **6.24** provide a detailed visual representation of the execution times recorded for each combination of workers based on the maximum depth level reached.

As can be seen, the **distribution of outliers follows a clear pattern**: although there are minor anomalies in levels 1 to 3, most outliers are **significantly concentrated in level 0**, corresponding to the download of the root URL. This concentration of highly variable download times at the initial level is the primary cause of the increase in the average described above for the "Depth 0" column in Table **6.7**.

Temporal Test Results

The methodology employed in the Bulk Test was reapplied to measure the average download time and the times for each depth level reached. Table **6.8** shows the values recorded for these measurements during the various runs.

			Depth 0		Depth 1		Depth 2		Depth 3	
	Global Mean	**Global Std**	**Mean**	**Std**	**Mean**	**Std**	**Mean**	**Std**	**Mean**	**Std**
1D - 5W	2.2603	0.3191	2.2245	0.8102	2.1417	0.2532				
1D - 10W	2.1439	0.2472	12.6965	12.0667	2.1755	0.2875				
2D - 5W	2.2886	0.2220	9.8882	14.1072	2.3351	0.3000	2.2616	0.2256		
2D - 10W	2.1641	0.1919	18.1471	17.5974	2.4698	0.6265	2.1508	0.2462		
3D - 5W	2.2166	0.1302	15.9283	11.4798	2.4357	0.6242	2.2799	0.1947	2.1736	0.1171
3D - 10W	2.2325	0.2370	18.3783	11.2602	2.5685	0.8457	2.4075	0.4614	2.1217	0.2733

Table 6.8: Temporal Test download time results for Serverless Crator.

As shown in the table, the average download time is **very stable** and remains around **2s**. However, the anomalies already found in the Bulk Test are also evident here, particularly at level 0. In these measurements, **these anomalies appear more frequently**, with average peaks reaching up to **18s**. These high values are caused by some runs that recorded a much longer download time. This is confirmed by the **high standard deviation value**, which indicates a **high variability in the data**.

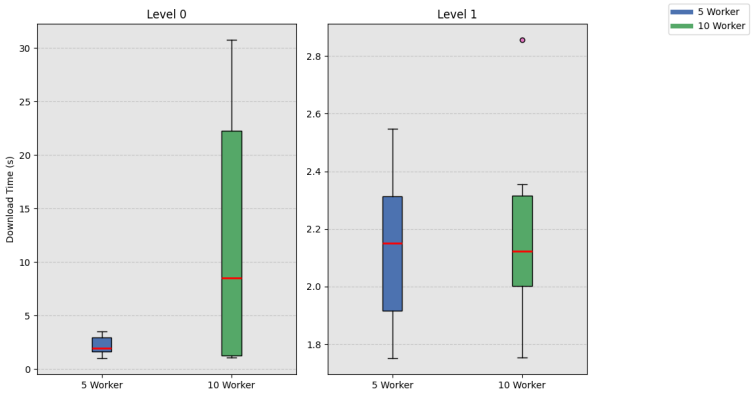

Figure 6.25: Temporal Test download time of Serverless Crator at level 1.

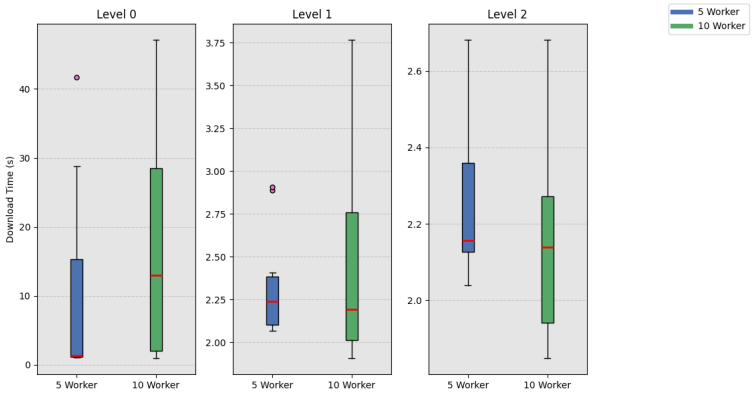

Figure 6.26: Temporal Test download time of Serverless Crator at level 2.

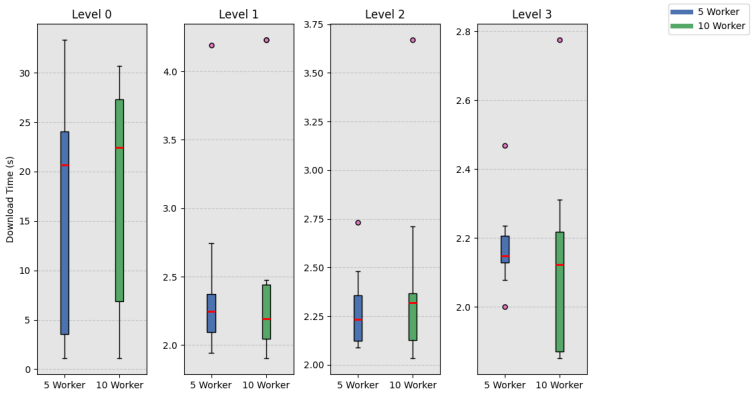

Figure 6.27: Temporal Test download time of Serverless Crator at level 3.

Figures **6.25**, **6.26** and **6.27** offer a better visualization of anomalies. In fact, it is possible to see how some very high download times, which are represented as outliers in the box plot, alter the average for the depth level to which they refer.

6.3 RQ3: Open-Source and Browser-Accessible Tool

This section tackles the horizontal scalability and accessibility of the tool, evaluating the dynamic management of URL queues and explaining how users can trigger the distributed crawling processes securely.

6.3.1 Queue Management and Time-to-Download

Bulk Test Results

The second metric introduced in this experiment is Time to Download (**TTD**). This metric is crucial for understanding the overall longer execution times compared to the on-premise version, as it provides us with a direct view of the critical issues of the serverless crawler.

TTD records the *exact time between when a link is added to the URL frontier and when it is extracted for download by the worker*. Analyzing this interval is essential for **identifying bottlenecks** caused by latency and **queue management** in the cloud environment. Table **6.9** shows in detail, as for the previous metric, the average values recorded both globally and for each depth level.

			Depth 0		Depth 1		Depth 2		Depth 3	
	Global Mean	**Global Std**	**Mean**	**Std**	**Mean**	**Std**	**Mean**	**Std**	**Mean**	**Std**
1D - 5W	58.1369	7.8523	2.5462	1.9585	58.7750	7.8619				
1D - 10W	46.9880	4.9705	4.5387	6.8212	47.4715	4.8838				
2D - 5W	740.6735	32.6413	8.4272	11.0946	474.0015	13.4383	809.7815	40.0905		
2D - 10W	635.6558	33.8010	5.6130	7.0717	447.4189	20.1672	682.1132	40.3208		
3D - 5W	3896.6851	401.623	6.1854	7.3612	495.6123	14.8307	2671.0921	155.6722	4561.2642	502.1243
3D - 10W	2764.3930	206.3122	5.2606	8.2388	446.5140	18.6706	2385.1296	111.5810	3141.3379	209.1791

Table 6.9: Bulk Test time-to-download results for Serverless Crator.

As shown in the table, there is a clear **upward trend** in the average TTD **as depth levels increase**.

This increase is not due to workers becoming slower, but is a direct effect of **URL frontier management**: newly discovered links are analyzed and queued, rather than simply being discarded. The continuous addition of new links to the queue inevitably

increases the average time a URL must wait before being extracted and processed.

This behavior is **particularly pronounced at greater depths**, where the accumulation of links results in average waiting times of around **4000s**.

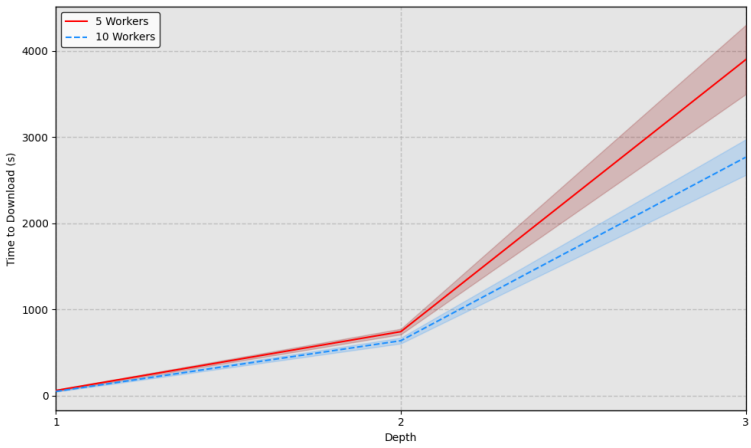

Figure 6.28: Bulk Test time-to-download of Serverless Crator.

As shown in Figure **6.28**, the **impact of worker configuration** is **significant**: as their number increases, there is a tangible decrease in Time to Download (TTD). This improvement in queue latency becomes particularly relevant as scan depth increases, demonstrating the effectiveness of parallelization.

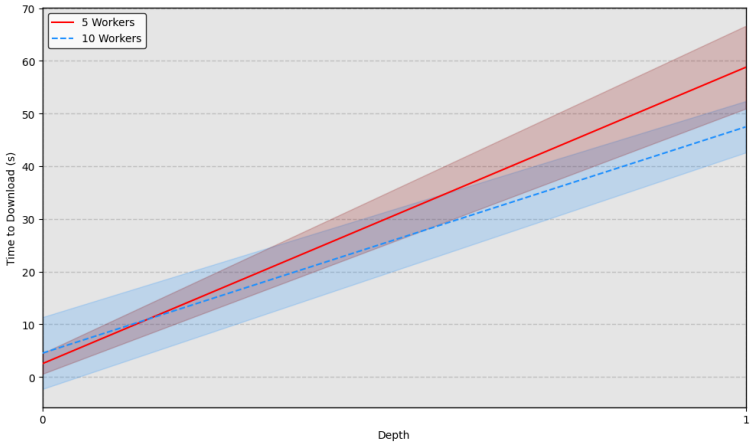

Figure 6.29: Bulk Test time-to-download of Serverless Crator at level 1.

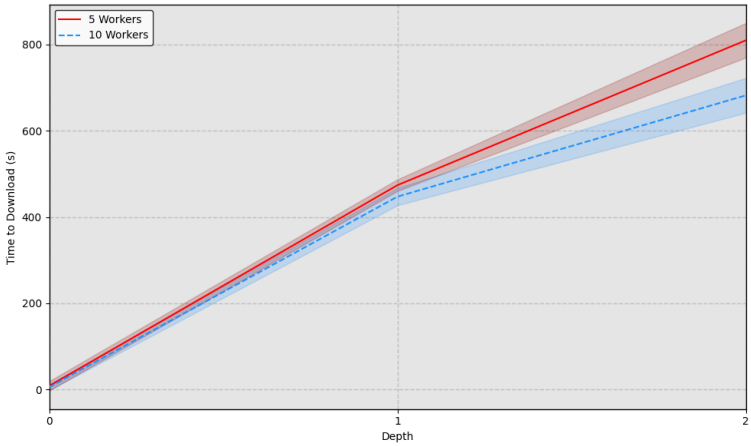

Figure 6.30: Bulk Test time-to-download of Serverless Crator at level 2.

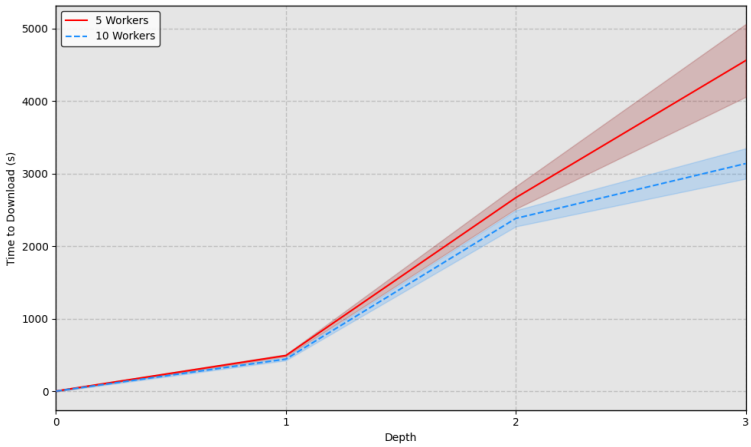

Figure 6.31: Bulk Test time-to-download of Serverless Crator at level 3.

Figures **6.29**, **6.30**, and **6.31** show this performance boost in detail: at the first few levels of depth, the difference in TTD between the different worker configurations is minimal. However, the gap gets way bigger as the scan depth increases, confirming that the effectiveness of the scalability offered by serverless architecture really shines in scenarios with higher load and complexity.

Temporal Test Results

Similar to download time, TTD was also measured by calculating the overall average times and the average times for each depth level. These values can be found in table **6.10**.

			Depth 0		Depth 1		Depth 2		Depth 3	
	Global Mean	**Global Std**	**Mean**	**Std**	**Mean**	**Std**	**Mean**	**Std**	**Mean**	**Std**
1D - 5W	58.3881	9.3412	2.5279	2.8382	55.7333	9.4792				
1D - 10W	45.2095	3.2669	4.4076	6.1899	87.7726	118.5731				
2D - 5W	734.6760	63.3996	1.8934	1.2966	463.0355	23.0423	795.3119	75.2776		
2D - 10W	663.4623	26.8006	1.6267	0.4838	446.4037	8.1701	738.5315	43.6057		
3D - 5W	3859.8249	644.7969	7.8219	10.5206	504.4347	108.8970	2629.5260	250.4146	4277.6099	866.6656
3D - 10W	2935.1523	316.8992	7.0935	9.8281	453.0803	16.1784	2548.2740	204.9538	3275.2452	348.9099

Table 6.10: Temporal Test time-to-download results for Serverless Crator.

Again, the TTD shows an **upward trend** as the crawling depth increases. An interesting fact is that the times recorded are **slightly lower** than those of the Bulk Test. This confirms that the tool is more effective in terms of time when used over longer periods. In addition, performance improvements are noticeable when using more workers, which significantly reduce the TTD.

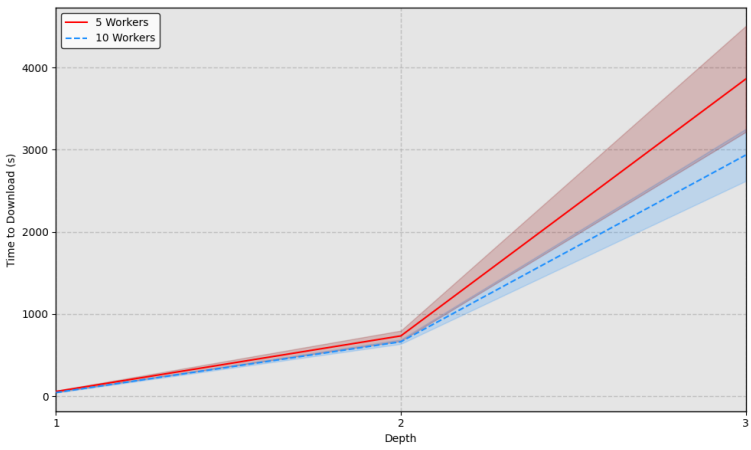

Figure 6.32: Temporal Test time-to-download of Serverless Crator.

Figure **6.32** show this upward trend showing that the number of workers reduces significantly the TTD.

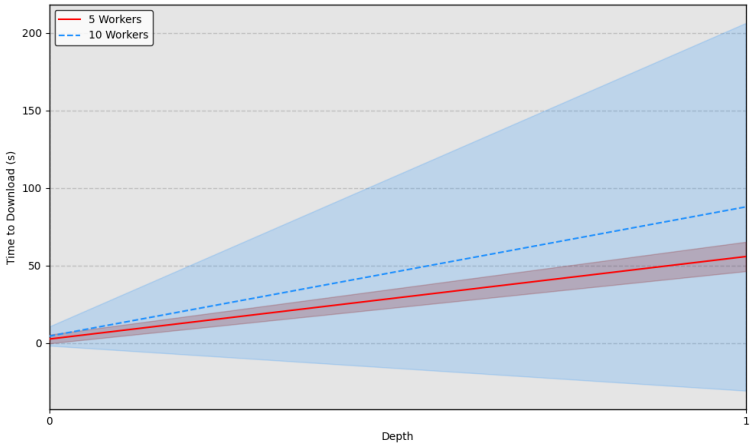

Figure 6.33: Temporal Test time-to-download of Serverless Crator at level 1.

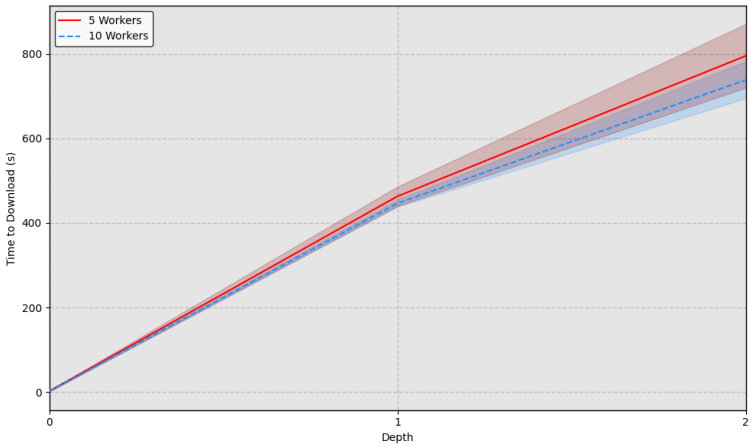

Figure 6.34: Temporal Test time-to-download of Serverless Crator at level 2.

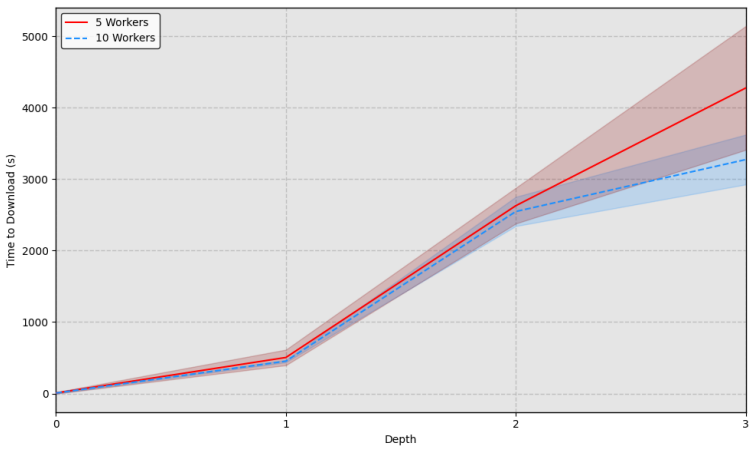

Figure 6.35: Temporal Test time-to-download of Serverless Crator at level 3.

Figures **6.33**, **6.34** and **6.35** show the difference in terms od TTD between the

configuration with 5 and 10 workers, although in this case the difference is less obvious.

6.4 Cost Analysis

As part of this experimentation, it is essential to conduct a detailed analysis of the **Operating Costs** (**OpEx**) associated with the usage of Microsoft Azure infrastructure. For these types of projects, cost analysis is crucial because it is required to:

- Assess the economic effectiveness of the cloud solution.
- Support arguments regarding financial and operational trade-offs in the migration from on-premises infrastructure to a cloud solution, in this case Microsoft Azure.

Expenditures will be presented in a **structured manner** to ensure a comprehensive understanding of the consumption:

- **Total Expenses**: daily and cumulative total costs incurred on the Azure infrastructure will be presented.
- **Service Costs**: costs will be further broken down for each individual Azure service used, showing specific daily and cumulative expenses.
- **Comparison and Break-even Analysis**: a direct comparison will be made between the actual costs of Azure and the estimated costs of an on-premises environment. Furthermore, a **Break-even Point** (BEP) analysis will be conducted to identify the operational threshold—in terms of number of executions—at which the initial investment for dedicated hardware becomes more convenient than the recurring costs of the serverless cloud solution.

This approach will provide a precise quantification of the value and economic impact of cloud migration. In addition, a section on **financial sustainability options** for the cloud solution will be included, taking into account both the expenses that will be discussed in detail in the following subsections and potential solutions for generating profits to cover the use of the services.

6.4.1 Total Expenses

Cost data presented in this and following subsections refer to the testing period conducted from **October 1, 2025**, to **November 9, 2025**. Therefore, all costs and related currency conversions are based on list prices and exchange rates in effect during that time period.

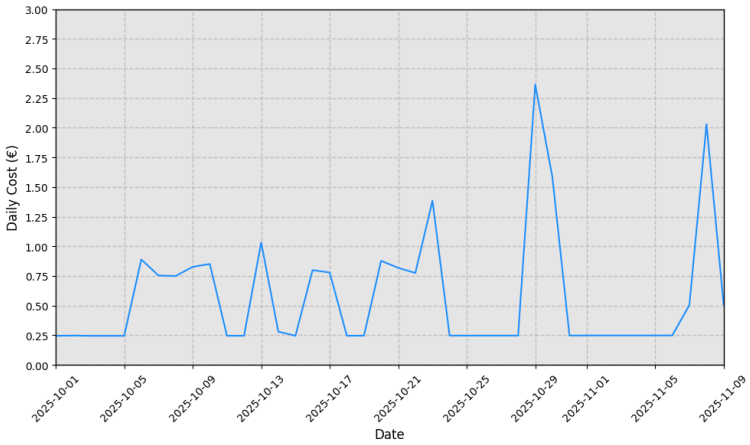

Figure 6.36: Daily costs.

As shown in figure **6.36**, **daily costs** for using the platform **proved to be low**. Average daily cost over the entire observation period were **€0.58/day**. This average was calculated by including both days of activity and days of "inactivity."

Tests were performed on the following dates:

- **Bulk Test**: October 23, October 29, October 30, and from November 7 to November 9.
- **Temporal Test**: from October 6 to October 10, October 16, October 17, and from October 20 to October 22.

If we exclude the idle days, when only the fixed costs of maintaining certain services were met, which amount to around **€0.28/day**, the average daily cost related only to the days when the platform were actually used is significantly higher, reaching **€1.03/day**.

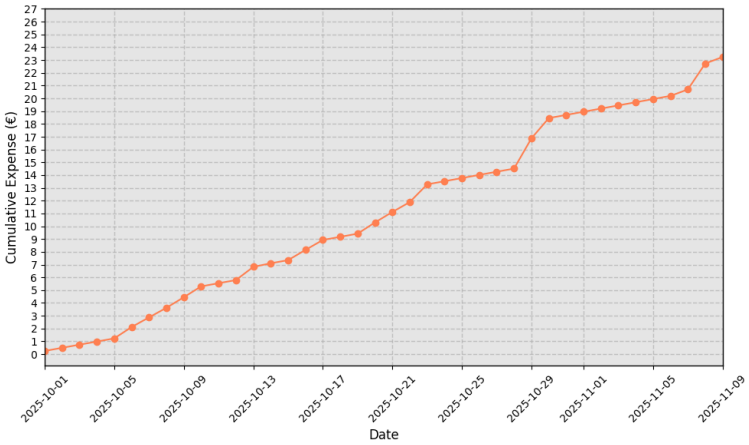

Figure 6.37: Cumulative costs.

A closer look at figure **6.37** reveals the real impact of the expenses incurred, illustrating the cumulative expenses recorded during the period analyzed.

It should be noted that **the total accumulated expenditure amounted to €23.23**, a number that is particularly significant when compared to the 120 runs performed overall. This calculation establishes an exceptionally **low average cost** per run, equal to approximately **€0.19/run**, if we consider the entire period of use of the cloud platform.

6.4.2 Service Costs

This subsection is designed to provide a more **in-depth analysis** of the cost structure in order to identify the specific services that represented the **biggest drivers of expenditure** during the period in question. The main purpose is to **quantify** the exact **percentage impact** of each service on the overall calculation. Through this measurement, it will be possible to clearly establish which services are the *Top-Spenders*, providing the necessary information base to guide and optimize future budget decisions and any cost containment strategies.

Cost identification was performed using the Azure platform's dedicated cost analysis tool. The services that affected spending were **Azure Container Registry**, **Azure Storage**, **Azure Virtual Network**, **Azure Container Apps** and **Azure Cosmos DB**.

It is important to note that although services such as Azure App Service, Azure

Load Balancer, Bandwidth, Azure Functions and Azure Log Analytics were identified, their cost **remained consistently 0**, so they do not appear in the total calculation.

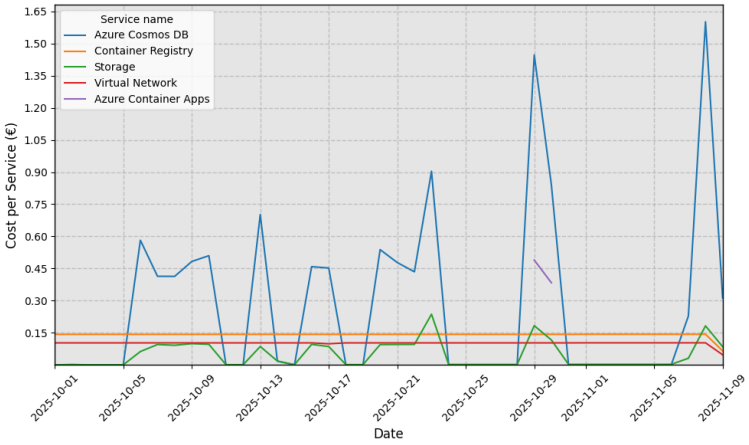

Figure 6.38: Daily service costs.

As shown in figure **6.38**, expenditures were **particularly consistent** for Azure Container Registry, Azure Storage, Azure Virtual Network, and Azure Container Apps. In contrast, Azure Cosmos DB showed **considerable variability** in costs. This fluctuation is directly **related to its pricing plan**, which charges a specific fee per million request units, reaching **peak costs of over €1.50**.

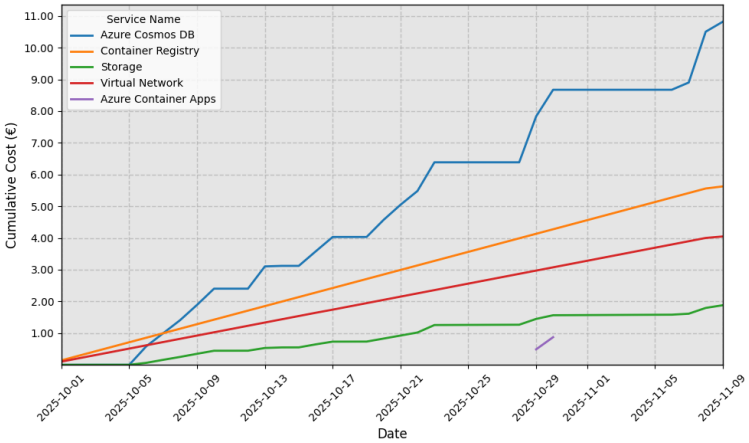

Figure 6.39: Cumulative services costs.

A visual analysis provided by figure **6.39** clearly shows that **Azure Cosmos DB** is the service that **has had the greatest impact** on overall costs. This impact is attributable to its **intensive use** for tracking data and the *history* associated with each

URL in every run performed, as well as the frequent need for queries to retrieve and insert new information.

Next, in terms of expenditure, we find Azure Container Registry and Azure Virtual Network. Both were essential: the first for managing and creating the TOR container used for anonymous requests, and the second for controlling incoming and outgoing network traffic.

Finally, **Azure Storage proved to be the most economical service**. Its use was limited to creating the queues necessary for managing the URL frontier and saving the data collected during each crawling session.

Service Name	Total Cost (€)	Percentage
Azure Container Apps	0.872584	3.75
Azure Cosmos DB	10.814993	46.53
Azure Container Registry	5.625379	24.20
Azure Storage	1.879424	8.09
Azure Virtual Network	4.047585	17.42

Table 6.11: Cumulative costs and percentage for each service.

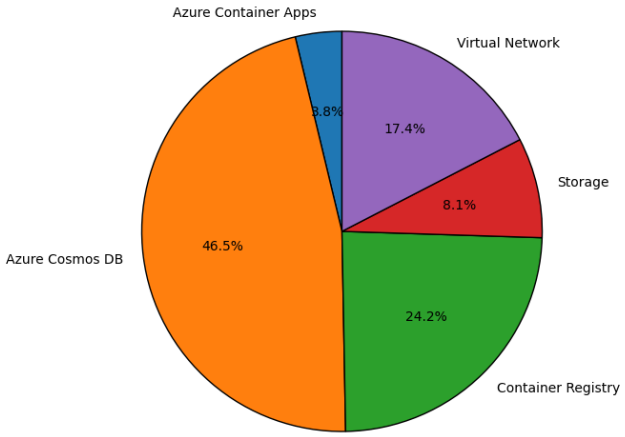

Figure 6.40: Percentage of cost per service.

Table **6.11** and figure **6.40** provide a comprehensive overview of the percentage breakdown of costs for each service.

6.4.3 Estimated on-premise costs

Following the analysis of cloud service costs, an estimation of the necessary on-premise expenses must be conducted to achieve equivalent performance locally with on-premise crawlers. This comparison is essential for evaluating the cost-effectiveness and convenience of migrating to a cloud infrastructure.

The following factors were taken into account when calculating on-premise costs:

- **Operating Costs**, namely the cost of electricity (**€/kWh**).
- **Capital Costs**, related to the depreciation of the hardware used.
- **VPN Costs**, incurred to ensure secure connections to the Tor network.

Operating Costs

The operating costs of on-premise executions are primarily related to the energy consumption of the system used. To calculate it, the following formula is applied:

$$C = P \times T \times €/\text{kWh},$$

where C is the operating cost, P is the power consumed by the system (expressed in kW), T is the total execution time (expressed in hours), and €/kWh is the cost of electricity. For a set of runs characterized by an average time per run t_{run} and a total number of runs N, the total usage time is:

$$T = t_{\text{run}} \times N.$$

The average time per run t_{run} and the number of runs N used in this calculation were taken from table **6.5**, whose data refer to the paper by De Pascale *et al.* [5].

Data and assumptions taken into account The system specifications used refer to those already explained in section **5.1.2**. Since direct power consumption measurements were not available, the absorbed power was estimated based on typical values of the main components:

- **CPU i7-9700**: with a TDP of 65 W, as reported in the technical specifications.[1]. Under intermediate load conditions (around 50%), actual consumption typically ranges between 40 and 60 W, while under maximum load it can reach values between 70 and 90 W.
- **RAM 16GB DDR4**: approximately 4–6 W, considering that the average consumption of an 8GB module is about 3 W.
- **Motherboard and chipset**: between 20 W and 40 W.
- **Storage (SSD/HDD)**: about 3–5 W.
- **Fans**: around 2–5 W.
- **Power supply unit**: efficienza stimata 85%.

The total consumption of the system is therefore given by the sum of the fixed components (P_{fixed}) and the CPU P_{CPU}. Two operating scenarios are considered for the CPU: 50% utilization and 100% utilization:

$$P_{\text{tot},50\%} = P_{\text{fixed}} + P_{\text{CPU},50\%} = (29\text{–}56)\,\text{W} + (40\text{–}60)\,\text{W} = 69\text{–}116\,\text{W}$$

$$P_{\text{tot},100\%} = P_{\text{fixed}} + P_{\text{CPU},100\%} = (29\text{–}56)\,\text{W} + (70\text{–}90)\,\text{W} = 99\text{–}146\,\text{W}$$

Considering also the efficiency η of the power supply:

$$P_{50\%} = \frac{P_{\text{tot},50\%}}{\eta} = \frac{69\text{–}116\,\text{W}}{0.85} = 81\text{–}136\,\text{W}$$

$$P_{100\%} = \frac{P_{\text{tot},100\%}}{\eta} = \frac{99\text{–}146\,\text{W}}{0.85} = 117\text{–}172\,\text{W}$$

To simplify calculations, an estimated average power of 0.1085 kW is used for the 50% load and 0.1445 kW for the 100% load. Table **6.12** provides a summary of the calculated power values.

CPU Load	Estimated Minimum Consumption	Estimated Maximum Consumption	Estimated Mean Consumption
50%	0,081 kW	0,136 kW	0,1085 kW
100%	0,117 kW	0,172 kW	0,1445 kW

Table 6.12: Estimated power consumption with on-premise hardware.

[1] `https://www.intel.com/content/www/us/en/products/sku/191792/intel-core-i79700-processor-12m-cache-up-to-4-70-ghz/specifications.html`

The total time T, calculated based on the previously reported data, is 1733 seconds for depth level 1, 9392 seconds for depth level 2, and 34206 seconds for depth level 3. The overall sum is therefore 45,331 seconds, which corresponds to 12.59 hours.Table **6.13** summarizes this values.

Depth	Time (s)
1	1733
2	9392
3	34206
Total	45331 (12.59 h)

Table 6.13: Total execution time with on-premise hardware.

Because the actual cost per kWh is unknown, an estimated European-average value of 0.2872 €/kWh, including taxes, is used as a reference for calculations. This value is based on data from the Eurostat report [2].

Using the formula related to the calculation of the operating costs, we obtain the following results:

$$C_{50\%} = 0.1085\,\text{kW} \times 12.59\,\text{h} \times 0.2872\,€/\text{kWh} = 0.39\,€$$

$$C_{100\%} = 0.1445\,\text{kW} \times 12.59\,\text{h} \times 0.2872\,€/\text{kWh} = 0.52\,€$$

CPU Load	Average Power (kW)	Runtime (h)	Energy Consumption (kWh)	Estimated Cost (€)
50%	0.1085	12.59	1.365	0.39
100%	0.1085	12.59	1.818	0.52

Table 6.14: Estimated operating costs based on average power consumption and runtime.

Table **6.14** summarizes all the calculations done. These results indicate that, for a runtime of approximately 12.6 hours, the **estimated operational costs** remain **very low**, below 1 euro for both load scenarios.

[2] https://ec.europa.eu/eurostat/statistics-explained/SEPDF/cache/45239.pdf

Capital Costs

In order to run computational tasks locally, it is necessary to account for the capital costs associated with building a dedicated PC. In addition to the previously specified CPU, RAM, and operating system, the pc also includes other essential components which are all necessary for a fully functional setup. The estimated costs for each major component are summarized in table 6.15:

Component	Specifications/Notes	Minimum Price (€)	Maximum Price (€)
CPU	Intel Core i7-9700 (Used/Refurbished)	220	300
Motherboard	B365/H370/B360 Chipset (LGA 1151 v2)	60	120
RAM	16GB (2x8GB) DDR4 @ 2666MHz	50	80
CPU Cooler	Air Cooler (Entry Level)	15	30
Storage	500GB SSD (Base SATA or NVMe)	40	60
Power Supply (PSU)	450W - 550W (80+ Bronze)	45	70
Case/Frame	Open Bench Frame (Test Bench)	30	60
ESTIMATED TOTAL	**Complete Assembly**	**460**	**720**

Table 6.15: Estimated Costs for an i7-9700 Test Bench (Real Market/Refurbished Prices).

These estimates give a reasonable idea of the initial investment required to set up a local test bench. While the exact cost may vary depending on market prices and component availability, a budget between €460 and €720 should cover the assembly of a reliable desktop PC with the specified configuration.

VPN Costs

The costs for using the VPN refer to NordVPN's two-year *Basic* plan, which provides access to the essential VPN services. Table **6.16** lists the costs associated with this plan:

Parameter	EUR Value (net price)
Duration	24 mesi
Total price	€71.76
Average monthly cost	€2.99/month

Table 6.16: VPN costs related to on-premise hardware.

6.4.4 Break-even Analysis

To determine the economic sustainability of the proposed serverless architecture compared to a traditional on-premise solution, a **Break-even Point** (BEP) analysis was performed. This analysis aims to identify the *tipping point*, expressed in the number of crawling runs, where the cumulative cost of the cloud solution equals the initial investment (CAPEX) required for the on-premise hardware.

The comparison is based on the following variables derived from the previous sections:

1. **Variable Cost** (Cloud), the serverless architecture operates on a purely OPEX (Operating Expense) model. Based on the cumulative expenses shown in previous sections, the average cost per run is approximately €0.19.

2. **Fixed Cost** (On-Premise), the traditional solution requires an upfront CAPEX investment. As estimated in Table**6.15**, the cost for a suitable test bench ranges between €460 and €720.

3. **Marginal Costs**, the electricity cost for the on-premise solution is approximately €0.003 per run, which is negligible for this comparison and therefore omitted to simplify the model.

The Break-even Point is calculated using the formula:

$$BEP_{runs} = \frac{AvgCost_{Cloud}}{FixedCost_{OnPrem}}$$

Applying the values for both the minimum and maximum hardware configurations:

$$BEP_{min} = \frac{0.19€/\text{run}}{460€} \approx 2421\text{runs}$$

$$BEP_{max} = \frac{0.19€/\text{run}}{720€} \approx 3789\text{runs}$$

The analysis reveals that Serverless Crator remains the most cost-effective solution for up to approximately 2,400 to 3,800 executions. Considering a realistic usage scenario for a research-oriented or investigative crawler—for example, performing one full crawl per day, the break-even point would be reached in **6.6 years**, if we consider the best-case scenario with cheaper hardware, and **10.3 years**, if we consider the worst-case scenario with expensive hardware.

This result highlights also a significant advantage of the serverless architecture for discontinuous, on-demand, or research-based workloads. The *pay-as-you-go* model allows for immediate operation with zero upfront investment. On the other hand, the on-premise solution becomes economically advantageous only for massive and continuous crawling operations (i.e 24/7 scraping) where the volume of executions drastically reduces the impact of the initial hardware purchase.

CHAPTER 7

Conclusions

This thesis investigated the feasibility, design and evaluation of a distributed serverless architecture for crawling the dark web through the Tor network. Starting from the challenges posed by dark-web environments, such as high latency, instability of onion services, anonymity requirements and limited visibility of interconnections, the work aimed to determine whether a serverless approach could effectively support large-scale crawling activities in this context.

7.1 Answering the Research Questions

7.1.1 RQ1 - Feasibility of a Serverless Dark-Web Crawler

The results demonstrate that it is possible to design and deploy a distributed serverless architecture capable of crawling the dark web. Despite the intrinsic instability of onion services and the performance overhead introduced by Tor routing, the serverless implementation successfully:

- Discovered and processed dark-web pages across multiple BFS levels.
- Maintained comparable coverage to the on-premise implementation.

- Managed concurrency effectively through event-driven orchestration.
- Ensured anonymity through containerized Tor proxying and circuit rotation.

Therefore, RQ1 can be positively answered: a distributed serverless architecture for dark-web crawling is technically feasible and operationally viable.

7.1.2 RQ2 - On-Premise vs Serverless: Key Differences

The experimental results showed clear structural and operational differences. While the on-premise solution maintains an advantage in terms of pure execution speed and throughput, the serverless solution demonstrated excellent robustness and, above all, effective horizontal scalability: increasing the number of workers (from 5 to 10) significantly reduced Time-to-Download times.

In addition, the choice between one of the two models is strongly influenced by the cost structure. While the local solution involves fixed costs related to hardware, electricity, and VPN infrastructure, the serverless one adopts a pay-per-use model. Break-even analysis suggests that the latter is particularly advantageous for intermittent or variable workloads, where economic efficiency is combined with operational flexibility.

7.1.3 RQ3 - Open-Source and Browser-Accessible Tool

The proposed architecture demonstrates that it is possible to implement a distributed cloud-based platform that enables users to initiate and monitor dark-web crawling activities without local installation. Through asynchronous HTTP APIs and serverless orchestration, users can interact with the system directly from a web interface.

7.2 Threats to Validity

Interpretation of the results must take into account certain limiting factors that may have affected the consistency and generalized applicability of the study. These

threats to validity have been classified into three main categories: **internal**, **external** and **conclusion**.

7.2.1 Internal Validity

Internal validity refers to the extent to which the results accurately reflect the impact of the architectures analyzed, excluding extraneous variables.

In this study, the main challenge is the stochastic nature of the Tor network: fluctuations in circuit performance and node congestion can introduce variations in response times that are not attributable to the crawler's logic. Furthermore, in the serverless architecture, the cold start phenomenon of Azure Functions can generate isolated latencies during the initial scaling phase. To mitigate these effects, repeated tests were conducted in controlled configurations, allowing anomalies to be averaged out and the systemic behavior of the infrastructure to be isolated.

7.2.2 External Validity

Threats to external validity concern the possibility of extending these results to broader contexts. Since the experiments focused on a single .onion domain and a single cloud provider (Azure) in a specific region, performance could vary significantly on other platforms or with different on-premises hardware configurations. Although the prototype demonstrates the effectiveness of the model, applying the results to large-scale industrial scenarios or using alternative providers could present different cost and latency dynamics, influenced by the specific resource management policies of the various vendors.

7.2.3 Conclusion Validity

Finally, the validity of conclusions is challenged by the inherent instability of the Dark Web ecosystem. The statistical "noise" introduced by temporarily offline services or sudden network variations can make it difficult to identify definitive patterns. Despite the experimental precision and multiple iterations, the stochastic factors inherent in the Tor infrastructure cannot be completely eliminated, representing an

intrinsic margin of uncertainty in any measurement of this type.

7.3 Future Works

The results presented in this thesis lead to different research directions and architectural optimizations, ranging from improving cloud infrastructure to integrating advanced data analysis techniques.

7.3.1 Cloud Evolution and Multi-Vendor Benchmarking

A natural extension of this research involves a comparative analysis across different **Cloud Service Providers (CSPs)**. While Azure provided a robust environment for validating the system, implementing the framework on platforms such as **AWS (Lambda)** or **Google Cloud (Functions)** would allow for a quantitative assessment of discrepancies in latency, cold-start management and operational costs.

Furthermore, the architecture could evolve toward an even more granular model by leveraging native other orchestration services, such as *AWS Step Functions*, to manage complex, stateful workflows while further reducing manual coordination logic.

7.3.2 Frontier Optimization and Machine Learning

At the algorithmic level, the current management of the **URL frontier** could significantly benefit from the introduction of **Machine Learning** models. Implementing focused crawling strategies based on Machine Learning techniques would enable the system to prioritize the exploration of domains with a higher probability of containing relevant content, thereby optimizing resource consumption and enhancing the quality of the collected dataset. In this context, integrating advanced mechanisms for cookie and session management would be of particular interest to navigate Onion services that require deeper user interaction.

7.3.3 Data Analytics and Intelligence

Finally, extending the framework toward a dedicated **Data Analytics** layer represents a fundamental step in transforming the crawler into a comprehensive intelligence tool. Integrating pipelines for automated Named Entity Recognition (NER), language classification and sentiment analysis on the crawled pages would allow for real-time mapping of market dynamics and community behaviors within the Dark Web. Such tools, combined with multidimensional data visualization, would provide critical decision support for cybersecurity and the large-scale monitoring of illicit activities.

7.4 Final Remarks

The results demonstrate that serverless computing is not only compatible with the technical constraints of the Tor network but represents a **paradigm shift** in how decentralized and anonymous data collection can be managed.

The proposed architecture successfully addresses the inherent limitations of on-premise solutions by introducing **native elasticity** and high fault tolerance. Through the decoupling of components and the use of asynchronous message queues, the system proved capable of handling the volatility of the Dark Web ecosystem without the need for manual hardware provisioning. Operationally, the transition to a Function-as-a-Service (FaaS) model significantly reduces the maintenance burden, allowing researchers to focus on data logic rather than infrastructure stability.

From a performance perspective, the study confirms that despite the overhead introduced by cloud orchestration and cold-start latencies, the serverless approach maintains **competitive throughput** (pages-per-minute) compared to fixed-resource environments. Furthermore, the cost-benefit analysis highlights that for intermittent or variable workloads—typical of investigative and academic research—the pay-per-use model offers a more sustainable and scalable economic profile.

In conclusion, this work lays a solid foundation for a new generation of cybersecurity tools. By bridging the gap between cloud-native technologies and darknet intelligence, it demonstrates that it is possible to collect web data at scale in a resilient

and cost-effective manner. The frameworks and methodologies developed in this thesis provide a versatile foundation for future explorations in automated threat intelligence and systematic monitoring of illicit digital ecosystems.

Bibliography

[1] T. Berners-Lee, R. Cailliau, A. Luotonen, H. F. Nielsen, and A. Secret, "The world-wide web," *Communications of the ACM*, vol. 37, no. 8, pp. 76–82, 1994. (Citato alle pagine 1 e 6)

[2] S. Sjouwerman. (2017) Surface web vs. deep web vs. dark web: Differences explained. [Online]. Available: https://www.examplhttps://blog.knowbe4.com/what-is-the-difference-between-the-surface-web-the-deep-web-and-the-dark-web (Citato alle pagine 1 e 11)

[3] D. Journal. (2024) Web crawler vs. web scraper: What's the difference? [Online]. Available: https://medium.com/@datajournal/web-crawling-vs-web-scraping-8fedba9531fb (Citato alle pagine 1 e 2)

[4] P. Narayanan, R. Ani, and A. T. King, "Torbot: open source intelligence tool for dark web," in *Inventive Communication and Computational Technologies: Proceedings of ICICCT 2019*. Springer, 2020, pp. 187–195. (Citato alle pagine 2 e 30)

[5] D. De Pascale, G. Cascavilla, D. A. Tamburri, and W.-J. V. D. Heuvel, "Crator: a dark web crawler," *arXiv preprint arXiv:2405.06356*, 2024. (Citato alle pagine 2, 30, 34, 51, 55 e 85)

[6] Oxford Languages. World wide web. [Online]. Available: https://doi.org/10.1093/OED/9887854441 (Citato a pagina 6)

[7] T. H. Nelson, "Complex information processing: a file structure for the complex, the changing and the indeterminate," in *Proceedings of the 1965 20th national conference*, 1965, pp. 84–100. (Citato a pagina 6)

[8] A. Broder, R. Kumar, F. Maghoul, P. Raghavan, S. Rajagopalan, R. Stata, A. Tomkins, and J. Wiener, "Graph structure in the web," *Computer networks*, vol. 33, no. 1-6, pp. 309–320, 2000. (Citato a pagina 7)

[9] S. Brin and L. Page, "The anatomy of a large-scale hypertextual web search engine," *Computer networks and ISDN systems*, vol. 30, no. 1-7, pp. 107–117, 1998. (Citato a pagina 7)

[10] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bringing order to the web." Stanford infolab, Tech. Rep., 1999. (Citato a pagina 7)

[11] Wikipedia. Http. [Online]. Available: https://en.wikipedia.org/wiki/HTTP (Citato a pagina 8)

[12] MDN Web Docs. (2024) Connection management in http/1.x. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Connection_management_in_HTTP_1.x (Citato a pagina 9)

[13] Mike Linksvayer. (2005) Redefining light and dark. [Online]. Available: https://gondwanaland.com/mlog/2005/11/28/redefining-light-and-dark/ (Citato a pagina 10)

[14] Wikipedia. Surface web. [Online]. Available: https://en.wikipedia.org/wiki/Surface_web (Citato a pagina 10)

[15] M. de Kunder and T. University. The size of the world wide web (the internet). [Online]. Available: https://www.worldwidewebsize.com/showGraph.php?Searchengine=Google&corpus=0&days=365 (Citato a pagina 11)

[16] S. Raghavan and H. Garcia-Molina, "Crawling the hidden web," Stanford, Tech. Rep., 2000. (Citato a pagina 11)

[17] L. Grustniy. (2021) Darknet, dark web, deep web e surface web: qual è la differenza? [Online]. Available: https://www.kaspersky.it/blog/

deep-web-dark-web-darknet-surface-web-difference/23836/ (Citato a pagina 11)

[18] Wikipedia. Dark web. [Online]. Available: https://en.wikipedia.org/wiki/Dark_web (Citato a pagina 11)

[19] P. Biddle, P. England, M. Peinado, B. Willman *et al.*, "The darknet and the future of content distribution," in *ACM Workshop on digital rights management*, vol. 6, 2002, p. 54. (Citato a pagina 11)

[20] Tor Project Inc. Tor. [Online]. Available: https://www.torproject.org/ (Citato a pagina 11)

[21] S. Kaur and S. Randhawa, "Dark web: A web of crimes," *Wireless Personal Communications*, vol. 112, pp. 2131–2158, 2020. (Citato a pagina 11)

[22] P. F. Syverson, D. M. Goldschlag, and M. G. Reed, "Anonymous connections and onion routing," in *Proceedings. 1997 IEEE symposium on security and privacy (cat. no. 97CB36097)*. IEEE, 1997, pp. 44–54. (Citato a pagina 12)

[23] Wikipedia. Onion routing. [Online]. Available: https://en.wikipedia.org/wiki/Onion_routing (Citato alle pagine 12 e 14)

[24] Geek for Geeks. Onion routing. [Online]. Available: https://www.geeksforgeeks.org/onion-routing/ (Citato a pagina 13)

[25] R. Dingledine, N. Mathewson, P. F. Syverson *et al.*, "Tor: The second-generation onion router." in *USENIX security symposium*, vol. 4, 2004, pp. 303–320. (Citato alle pagine 14, 15 e 16)

[26] Tor Project Inc. Tor specifications. [Online]. Available: https://spec.torproject.org/tor-spec/creating-circuits.html (Citato a pagina 16)

[27] T. V. Udapure, R. D. Kale, and R. C. Dharmik, "Study of web crawler and its different types," *IOSR Journal of Computer Engineering*, vol. 16, no. 1, pp. 01–05, 2014. (Citato alle pagine 16, 17 e 18)

[28] M. A. Kausar, V. Dhaka, and S. K. Singh, "Web crawler: a review," *International Journal of Computer Applications*, vol. 63, no. 2, pp. 31–36, 2013. (Citato a pagina 17)

[29] S. Dhenakaran and K. T. Sambanthan, "Web crawler-an overview," *International Journal of Computer Science and Communication*, vol. 2, no. 1, pp. 265–267, 2011. (Citato a pagina 18)

[30] S. L. Garfinkel, *Architects of the Information Society: Thirty-Five Years of the Laboratory for Computer Science at MIT*, H. Abelson, Ed. Cambridge, MA: MIT Press, 1999. (Citato a pagina 20)

[31] J. McCarthy, "Time sharing computer systems," in *Management and the Computer of the Future*, M. Greenberger, Ed. Cambridge, MA: MIT Press, 1962, pp. 220–248. (Citato a pagina 20)

[32] Google Trends, "Google trends," 2025. [Online]. Available: https://trends.google.com/ (Citato alle pagine 20 e 24)

[33] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. H. Katz, A. Konwinski, G. Lee, D. A. Patterson, A. Rabkin, I. Stoica *et al.*, "Above the clouds: A berkeley view of cloud computing," Technical Report UCB/EECS-2009-28, EECS Department, University of California ..., Tech. Rep., 2009. (Citato alle pagine 20 e 23)

[34] Gartner. Cloud computing. [Online]. Available: https://www.gartner.com/en/information-technology/glossary/cloud-computing (Citato a pagina 20)

[35] P. Mell and T. Grance, "The nist definition of cloud computing," *National institute of science and technology, special publication*, vol. 800, no. 2011, p. 145, 2011. (Citato a pagina 21)

[36] T. Erl, Z. Mahmood, and R. Puttini, *Cloud Computing: Concepts, Technology & Architecture*. Upper Saddle River, NJ: Prentice Hall, 2013. (Citato alle pagine 21, 22 e 23)

[37] S. Susnjara and I. Smalley. (2024, June) What is serverless computing? [Online]. Available: https://www.ibm.com/think/topics/serverless (Citato a pagina 23)

[38] A. W. Services. (2014, November) Introducing aws lambda. [Online]. Available: https://aws.amazon.com/blogs/aws/introducing-aws-lambda/ (Citato a pagina 24)

[39] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar *et al.*, "Cloud programming simplified: A berkeley view on serverless computing," *arXiv preprint arXiv:1902.03383*, 2019. (Citato alle pagine 24 e 25)

[40] M. Stigler, *Beginning Serverless Computing: Developing with Amazon Web Services, Microsoft Azure, and Google Cloud*. Berkeley, CA: Apress, 2018. [Online]. Available: https://link.springer.com/book/10.1007/978-1-4842-3084-8 (Citato alle pagine 24 e 25)

[41] R. Baeza-Yates, C. Castillo, M. Marin, and A. Rodriguez, "Crawling a country: better strategies than breadth-first for web page ordering," in *Special interest tracks and posters of the 14th international conference on World Wide Web*, 2005, pp. 864–872. (Citato a pagina 27)

[42] C. Hsinchun, C. Yi-Ming, M. Ramsey, and C. C. Yang, "An intelligent personal spider (agent) for dynamic internet/intranet searching," *Decision Support Systems*, vol. 23, no. 1, pp. 41–58, 1998. (Citato a pagina 27)

[43] W. Wang, X. Chen, Y. Zou, H. Wang, and Z. Dai, "A focused crawler based on naive bayes classifier," in *2010 Third International Symposium on Intelligent Information Technology and Security Informatics*. IEEE, 2010, pp. 517–521. (Citato a pagina 28)

[44] P. Boldi, B. Codenotti, M. Santini, and S. Vigna, "Ubicrawler: A scalable fully distributed web crawler," *Software: Practice and Experience*, vol. 34, no. 8, pp. 711–726, 2004. (Citato a pagina 28)

[45] H.-T. Lee, D. Leonard, X. Wang, and D. Loguinov, "Irlbot: scaling to 6 billion pages and beyond," *ACM Transactions on the Web (TWEB)*, vol. 3, no. 3, pp. 1–34, 2009. (Citato a pagina 28)

[46] J. Stevenson. (2021) Scaling up a serverless web crawler and search engine. [Online]. Available: https://aws.amazon.com/blogs/architecture/scaling-up-a-serverless-web-crawler-and-search-engine/ (Citato alle pagine 29 e 34)

[47] D. Martinaitis. (2020) Serverless architecture for a web scraping solution. [Online]. Available: https://aws.amazon.com/blogs/architecture/serverless-architecture-for-a-web-scraping-solution/ (Citato a pagina 29)

[48] J. Bergman and O. B. Popov, "Exploring dark web crawlers: a systematic literature review of dark web crawlers and their implementation," *IEEE Access*, vol. 11, pp. 35 914–35 933, 2023. (Citato a pagina 30)

[49] Microsoft Corporation, "Microsoft Azure – Cloud Computing Services," https://azure.microsoft.com/, n.d. (Citato a pagina 39)

[50] deepweb. (2025) Cocorico. [Online]. Available: https://deepweb.net/catalog/xv3dbyx4iv35g7z2uoz2yznroy56oe32t7eppw2l2xvuel7km2xemrad.onion (Citato a pagina 55)

Acknowledgements

"La vita è piena di scelte e incroci. Siamo noi a decidere quale strada prendere."

Questa è la frase con cui ho concluso il testo della mia tesi triennale, ed è proprio da qui che voglio riprendere il filo di quanto scritto due anni fa. Sì, sono già volati altri due anni e mi sembra ieri di aver iniziato questo percorso di laurea magistrale dal quale ho imparato tanto sia dal punto di vista della formazione che da quello della mia crescita personale e maturità.

Per iniziare, ci tengo a ringraziare il Prof. Fabio Palomba, che grazie alla sua disponibilità e professionalità ha saputo guidarmi nel percorso di Erasmus dal quale ne è risultato dal punto di vista accademico questo lavoro di tesi che mi ha appassionato ed interessato fin dall'inizio.

Un ringraziamento speciale va alla Dott.ssa Jessica De Pascale del JADS, con la quale ho avuto il privilegio di realizzare questo lavoro. La sua profonda competenza negli ambiti del dark web e del crawling è stata una guida fondamentale, permettendomi di operare nelle migliori condizioni possibili. La sua straordinaria disponibilità e i suoi preziosi consigli sono stati determinanti per la riuscita di questo progetto.

Grazie a David, Dario e Luca: non avrei potuto chiedere compagni di viaggio migliori. Insieme abbiamo trasformato tre mesi all'estero in un'esperienza di vita

totale, fatta di crescita professionale, nuovi incontri e tantissimo divertimento. Pur nella diversità dei nostri caratteri, l'affinità è stata immediata e profonda. A loro si aggiungono Federico e Federica: è quasi incredibile essersi incontrati così lontano da casa per scoprire di vivere a pochissimi chilometri di distanza; il loro tocco di allegria e di sana follia è stato incredibile. Tutti voi siete stati il valore aggiunto di uno dei periodi più formativi e intensi del mio percorso magistrale.

Il ringraziamento più sentito e per il quale le sole parole di questa sezione non basterebbero mai va alla mia famiglia.
Ai miei nonni che mi sostengono e che con curiosità si interessano di ogni mia passione, dei miei obiettivi e di quello che mi piace. La vostra vicinanza e interesse per un mondo così lontano dai vostri tempi non ha prezzo e mi ritengo estremamente fortunato a poterla avere. Da oggi il vostro "tecnico" di fiducia adesso è ancora più qualificato rispetto a 2 anni fa e adesso non ci sono problemi informatici che non posso risolvere per voi.
Ai miei genitori che hanno continuato e continueranno a credere in me nelle mie scelte passate e future. A voi devo tutte le esperienze che ho vissuto fino ad ora. È grazie a voi ed al vostro sostegno se ho potuto "sperimentare" un possibile periodo di studio lontano da casa, se ho potuto vivere una delle esperienze all'estero più formative per una qualsiasi persona, e per tutto quello che mi insegnate e mi date ogni giorno. Spero di ripagarvi presto per tutti i sacrifici che avete fatto per me in tutti questi anni di istruzione e di esperienze meravigliose, rendendovi fieri dei traguardi e di quello che potrò diventare in futuro.
A mia zia Mariella, che conoscendo anche lei questa materia ha sempre saputo darmi i consigli giusti e che, molto spesso, ha esultato molto più di me per i miei traguardi accademici, facendomi capire quanto siano importanti. A te devo la passione e la curiosità che mi ha avvicinato a questo meraviglioso mondo in costante evoluzione.
Alla mia bellissima Grazia, ti ringrazio dal profondo del mio cuore per la pazienza, l'amore e il supporto che mi hai dato in questi anni. Rispetto a due anni fa sono cambiate tantissime cose, ci siamo laureati entrambi, abbiamo finito ed iniziato capitoli nuovi della nostra vita dal punto di vista accademico e lavorativo. Da quando sono partito per l'erasmus e da quando tu hai iniziato a lavorare fuori sento che il

nostro legame si sia rafforzato rispetto agli altri anni, perchè dalla lontananza siamo riusciti e stiamo riuscendo comunque a viverci quotidianamente, come se stessimo sempre vicini ogni giorno. Gli ultimi due anni sono a parer mio quelli più formativi per entrambi dal punto di vista della crescita personale e di coppia perchè abbiamo fatto lo step successivo e siamo passati alla vita lontano da casa e a quella lavorativa.

Un grazie immenso va ai miei amici di sempre.
Ad Anthony, che ho capito essere una persona incredibilmente simile a me dal punto di vista caratteriale e degli interessi, oltre che ad un vero amico su cui poter contare.
A michele, capace di strappare un sorriso a chiunque, ma dotato di una sensibilità rara nel comprendere gli altri.
Ad Armando e Roberto, che nonostante la lontananza per lavoro o studio continuiamo a vivere questa lunga amicizia sfruttando tutti i momenti utili per vederci, uscire e condividere quello che stiamo facendo.
A Giovanni, Luca e Federico, per la bellezza della vostra spontaneità e per tutti i momenti passati tra risate, giochi e lunghe tavolate.
A Gennaro e Marco, con cui il rapporto si è stretto e consolidato anno dopo anno, trasformandosi in una profonda amicizia e reciproca fiducia.
A Vincenzo, che dopo anni di conoscenza ho scoperto essere un vero amico e una persona fidata, molto seria e incredibilmente disponibile.
A tutti voi non potrò mai dirvi grazie a sufficienza per tutti i bei momenti passati insieme, per il supporto e l'aiuto che mi avete dato nei momenti più difficili e per tutto quello che abbiamo condiviso e che continueremo a condividere.

Infine, ricollegandomi alla citazione da cui sono partito, ogni persona incontrata in questi due anni è stata un fondamentale compagno di viaggio. Un ringraziamento sincero va quindi a tutti i colleghi conosciuti durante il percorso magistrale e alle persone incontrate in Erasmus: pur provenendo da realtà così diverse, ognuno di voi ha contribuito ad accrescere il mio bagaglio umano e culturale. Grazie per aver incrociato la mia strada e per aver reso questo cammino non solo una serie di scelte accademiche, ma un'esperienza di vita profonda e indimenticabile.

Per concludere, a differenza di quanto fatto nella tesi triennale, questa volta non ringrazierò me stesso. Preferisco usare quest'ultima sezione come un diario, uno spazio sospeso per riflettere su ciò che è stato. Ripensando a questi anni trovo incredibile il percorso compiuto e i traguardi raggiunti. Sembra ieri che varcavo per la prima volta la soglia dell'università eppure sono volati sei anni intensi, densi di vita. Ogni bivio che ho incontrato e ogni decisione che ho preso mi hanno portato a essere la persona che sono oggi: più sicura, più consapevole dei miei mezzi e delle mie capacità. Due anni fa concludevo il mio primo ciclo di studi chiedendomi, forse con un pizzico di timore, cosa mi avrebbe riservato il domani. Oggi quella prospettiva è cambiata radicalmente. Non ho più paura del futuro ma, al contrario, lo guardo con curiosità e fiducia, sapendo che ogni esperienza, ogni incontro e ogni imprevisto avrà una sua motivazione e un suo fine. L'augurio che faccio al me stesso di oggi è di non accontentarsi mai, di non restare statico, ma di abbracciare la vita con dinamismo, avendo il coraggio di stravolgerla quando serve per continuare a crescere. E al me stesso del futuro, che rileggerà queste righe tra un mese, un anno o molto più tempo: sii sempre fiero delle scelte che hai compiuto. Spero che i tuoi sogni si siano avverati, anche solo in parte; perché se sono rimasti quelli che coltivo oggi, saranno sicuramente meravigliosi.